VOL. 05

Vue

From your first template to components, routing and state

Source	vuejs.org
Generated	2026-10-10
Level	Beginner to intermediate
Estimated study time	25 hours
Language	English
Volume	Vol. 05 - Vue

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 8

Chapter map 8

Chapter 1 - What Vue is and your first application 9

1.1 Summary 9

1.2 Concepts 9

What Vue is 9

Where Vue fits 10

Two ways to run Vue 10

The shape of an application 10

The render loop 11

1.3 Examples 11

1.4 Common mistakes and best practices 13

1.5 Practice exercises 13

Solutions 13

1.6 Mini test 13

Answer key and explanations 14

Chapter 2 - Template syntax and directives 14

2.1 Summary 14

2.2 Concepts 15

Interpolation 15

Anatomy of a directive 15

The shorthands you will use all day 15

Conditional rendering: `v-if` versus `v-show` 15

List rendering 16

Event handling 16

Directives for special text 16

2.3 Examples 17

2.4 Common mistakes and best practices 19

2.5 Practice exercises 20

Solutions 20

2.6 Mini test 20

Answer key and explanations 20

Chapter 3 - Reactivity: ref() and reactive() 21

3.1 Summary . . . 21
3.2 Concepts . . . 21
What reactivity buys you . . . 21
`ref()` and `reactive()` side by side . . . 21
`.value` and unwrapping . . . 22
The limits of `reactive()` . . . 22
Shallow and frozen variants . . . 22
Batching and `nextTick()` . . . 22
3.3 Examples . . . 23
3.4 Common mistakes and best practices . . . 24
3.5 Practice exercises . . . 25
Solutions . . . 25
3.6 Mini test . . . 25
Answer key and explanations . . . 26
Chapter 4 - Computed properties and watchers . . . 26
4.1 Summary . . . 26
4.2 Concepts . . . 26
Computed property versus method . . . 26
Derived value, cached . . . 27
Watching state . . . 27
`watchEffect()` . . . 27
Choosing the right tool . . . 28
4.3 Examples . . . 28
4.4 Common mistakes and best practices . . . 30
4.5 Practice exercises . . . 31
Solutions . . . 31
4.6 Mini test . . . 31
Answer key and explanations . . . 32
Chapter 5 - Class, style bindings and transitions . . . 32
5.1 Summary . . . 32
5.2 Concepts . . . 32
Class binding . . . 32
Style binding . . . 33
The `` classes . . . 33
Transitions need a single child . . . 33

``. . . 33
5.3 Examples . . . 33
5.4 Common mistakes and best practices . . . 37
5.5 Practice exercises . . . 37
Solutions . . . 37
5.6 Mini test . . . 38
Answer key and explanations . . . 38
Chapter 6 - Forms with v-model . . . 38
6.1 Summary . . . 38
6.2 Concepts . . . 39
What each control binds . . . 39
Modifiers . . . 39
The same sugar, written by hand . . . 39
A controlled custom component . . . 39
6.3 Examples . . . 39
6.4 Common mistakes and best practices . . . 42
6.5 Practice exercises . . . 43
Solutions . . . 43
6.6 Mini test . . . 43
Answer key and explanations . . . 43
Chapter 7 - Components and props . . . 44
7.1 Summary . . . 44
7.2 Concepts . . . 44
Anatomy of a component . . . 44
Declaring props . . . 44
One-way data flow . . . 45
Attribute inheritance . . . 45
`defineExpose` and template refs . . . 45
7.3 Examples . . . 45
7.4 Common mistakes and best practices . . . 48
7.5 Practice exercises . . . 49
Solutions . . . 49
7.6 Mini test . . . 49
Answer key and explanations . . . 50
Chapter 8 - Events, slots and provide/inject . . . 50

8.1 Summary . . . 50
8.2 Concepts . . . 50
Events flow upward . . . 50
`v-model` on a component, spelled out . . . 51
Slots at a glance . . . 51
provide and inject . . . 51
8.3 Examples . . . 51
8.4 Common mistakes and best practices . . . 55
8.5 Practice exercises . . . 55
Solutions . . . 56
8.6 Mini test . . . 56
Answer key and explanations . . . 56
Chapter 9 - Lifecycle hooks, template refs and composables . . . 56
9.1 Summary . . . 56
9.2 Concepts . . . 57
The lifecycle in order . . . 57
Template refs . . . 57
Composables . . . 57
9.3 Examples . . . 58
9.4 Common mistakes and best practices . . . 61
9.5 Practice exercises . . . 61
Solutions . . . 61
9.6 Mini test . . . 61
Answer key and explanations . . . 62
Chapter 10 - Single-file components and build tooling . . . 62
10.1 Summary . . . 62
10.2 Concepts . . . 62
The three blocks of an `.vue` file . . . 62
A project layout . . . 63
What Vite does for you . . . 63
10.3 Examples . . . 63
10.4 Common mistakes and best practices . . . 66
10.5 Practice exercises . . . 66
Solutions . . . 66
10.6 Mini test . . . 66

Answer key and explanations . . . 67
Cheat sheet . . . 67
11.1 Template directives . . . 67
11.2 Composition API . . . 68
11.3 Components and communication . . . 68
11.4 Lifecycle . . . 69
11.5 Project commands . . . 69
11.6 Router essentials . . . 69
11.7 Pinia essentials . . . 69
Final test . . . 70
12.1 Final test answers . . . 71
Sources . . . 72
Chapter 11 - Vue Router, Pinia and data fetching . . . 73
14.1 Summary . . . 73
14.2 Concepts . . . 73
Wiring the router . . . 74
Reading and moving . . . 74
Guards . . . 74
Pinia in one paragraph . . . 74
Fetching with states . . . 74
14.3 Examples . . . 75
14.4 Common mistakes and best practices . . . 78
14.5 Practice exercises . . . 79
Solutions . . . 79
14.6 Mini test . . . 79
Answer key and explanations . . . 80
Chapter 12 - Testing, TypeScript and the mini project . . . 80
15.1 Summary . . . 80
15.2 Concepts . . . 80
The testing tools . . . 80
TypeScript in components . . . 80
The mini project shape . . . 81
15.3 Examples . . . 81
15.4 Common mistakes and best practices . . . 86

15.5 Practice exercises 87
Solutions 87
15.6 Mini test 87
Answer key and explanations 88

How to use this volume

Prerequisites: HTML (Vol. 01) and basic JavaScript (Vol. 03). You should be comfortable with `let` and `const`, functions and arrow functions, template literals, array methods such as `map` and `filter`, and the idea of modules (`import` / `export`). No build tool experience is assumed; Chapter 10 installs the tooling with you.

Estimated study time: about 25 hours, spread over 12 content chapters. Vue has a small core and a gentle learning curve, so most of the time goes into the hands-on examples rather than into memorising rules.

How the chapters work: every chapter follows the same shape. A summary tells you the ideas in one glance; the concepts section explains them with tables and diagrams; the examples section runs code and shows the real result; the mistakes table collects what usually goes wrong; and each chapter closes with practice exercises, solutions, and a mini test with an answer key. Read with an editor open and type the examples yourself.

Which code blocks are meant to run. Blocks tagged `html` are complete pages that open directly in a browser. Blocks tagged `js` that begin with `import { ... } from "vue"` run in Node once you have installed the `vue` package. Blocks tagged `vue` are single-file components and need the dev server that Chapter 10 sets up. Blocks tagged `jsref` are API listings, not programs. Rendered results are shown as `html`; console results are shown as `text`.

How to run the code

shell

```shell
# 1. reactivity-only examples (chapter 3 and 4) run in Node
npm init -y && npm i vue
node example.js

# 2. scaffold a real project once, then use it for every later chapter
npm create vue@latest
cd <project-name> && npm install && npm run dev   # serves http://localhost:5173

# 3. the plain html pages from chapter 1 and 2 only need a browser
start index.html
```

Chapter map

#	Chapter	You will learn
1	What Vue is and your first application	the framework, mounting, state, rendering
2	Template syntax and directives	interpolation, `v-bind`, `v-if`, `v-for`, `v-on`
3	Reactivity: `ref()` and `reactive()`	tracked state, `.value`, deep and shallow
4	Computed properties and watchers	derived values, caching, side effects
5	Class, style bindings and transitions	dynamic styling, animated enter and leave
6	Forms with `v-model`	inputs, checkboxes, radios, selects, modifiers

#	Chapter	You will learn
7	Components and props	defining components, passing data down
8	Events, slots and provide/inject	talking upward, flexible layouts, shared values
9	Lifecycle hooks, template refs and composables	timing, DOM access, reusable logic
10	Single-file components and build tooling	SFCs, Vite, dev server, production build
11	Vue Router, Pinia and data fetching	pages, stores, loading and error states
12	Testing, TypeScript and the mini project	Vitest, typed components, a task app
13	Cheat sheet	compact reference
14	Final test	25 questions with answers

Chapter 1 - What Vue is and your first application

1.1 Summary

- Vue is a JavaScript framework for building user interfaces out of components.
- One Vue application owns a single DOM element and everything inside it.
- A component is a template plus state plus behaviour, stored together.
- You can start with one `<script>` tag from a CDN, or with Vite and single-file components.
- Vue compiles templates into render functions and patches only the parts of the DOM that change.
- State is plain data; when it changes, the view follows automatically.
- Vue is called progressive because you can adopt it for one widget or for a whole application.
- Routing (Vue Router) and state management (Pinia) are separate official libraries you add only when you need them.
- Data flows one way: parents pass props down, children raise events up.
- The modern way to organise component logic is the Composition API with `<script setup>`.

1.2 Concepts

What Vue is

Vue (pronounced like the English word "view") lets you declare what the interface should look like for a given state, instead of writing step-by-step DOM instructions. You write a template - mostly ordinary HTML with a few extra attributes - bind it to data, and Vue keeps the two in sync. The version used in this volume is Vue 3.

Two ideas do most of the work:

1. Templates are declarative. You describe the output for a state; you never say "find this paragraph and change its text".
2. State is reactive. When a reactive value changes, every place that reads it is re-evaluated and only the real DOM nodes that actually differ are updated.

Where Vue fits

Approach	Interface unit	Routing and state
Vue	HTML-based templates with directives	Official Router and Pinia packages
React	JSX written inside JavaScript functions	Community packages (React Router and others)
Angular	HTML templates plus TypeScript decorators	Router and forms included in the core
Vue with no framework	-	fine for static pages; you hand-write every update

Vue sits deliberately between "just the view layer" and "a complete platform": strong opinions about components and reactivity, optional packages for routing, state management and testing.

Two ways to run Vue

Way	What you need	Use it for
Global CDN build	an HTML file and a browser	learning, prototypes, embedded widgets
Vite + single-file components	Node.js, `npm create vue@latest`	real applications and team projects

The CDN build loads the whole library as one script and exposes a global `Vue` object. The project setup compiles `.vue` single-file components, serves them with a dev server, and bundles them for production.

The shape of an application

output

```
index.html          main.js               App.vue
+-----------+     +----------------+     +--------------------+
| <div      | --> | createApp(App) | --> | <template> ...     |
|  id="app">|     |   .mount()     |     | <script setup> ...|
| </div>    |     +----------------+     | <style scoped> ...|
+-----------+                            +--------------------+
```

`main.js` is the entry point: it imports the root component, creates the application object, and mounts it on a real DOM element. Everything below that element belongs to Vue from then on.

The render loop

output

```
         reads state
 template  <----------  component state
    |                        |
    v                        |
  real DOM  -- events -->  methods
  (updated by Vue)          (change state)
```

Events call methods, methods change state, and the template re-renders. You rarely touch the DOM yourself.

1.3 Examples

Example 1: a page with no tools at all.

HTML

```html
<!doctype html>
<html>
  <head>
    <title>First Vue app</title>
  </head>
  <body>
    <div id="app">
      <h1>{{ message }}</h1>
    </div>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
    <script>
      const { createApp } = Vue;
      createApp({
        data() {
          return { message: "Hello, Vue" };
        }
      }).mount("#app");
    </script>
  </body>
</html>
```

Rendered output:

HTML

```html
<div id="app">
  <h1>Hello, Vue</h1>
</div>
```

The `{{ message }}` placeholder is replaced by the value from `data()`. Note that the element you mount on stays in the page; only its contents are replaced.

Example 2: a component that reacts to a click.

JavaScript

```
Vue.createApp({
  data() {
    return { count: 0 };
  },
  methods: {
    increment() {
      this.count = this.count + 1;
    }
  }
}).mount("#app");
```

HTML

```
<div id="app">
  <button @click="increment">Clicked {{ count }} times</button>
</div>
```

Rendered output after two clicks:

HTML

```
<div id="app">
  <button>Clicked 2 times</button>
</div>
```

The script block only describes state and behaviour. The template binds the click to `increment`, and Vue re-renders the text when `count` changes.

Example 3: the application object itself. The block below is a listing, not a program.

JSREF

```
const app = Vue.createApp(rootComponent)  // build an application from a root component
app.mount("#app")                         // render it into a real DOM element
app.component("MyButton", definition)     // register a component everywhere
app.use(plugin)                           // install a plugin: router, pinia, ...
app.provide(key, value)                   // a value every descendant can inject
app.config.errorHandler = function (err)  // one place to log render errors
```

Example 4: the same counter, written the modern way. This file is a single-file component; it needs the dev server from Chapter 10.

Vue

```
<script setup>
import { ref } from 'vue'

const count = ref(0)

function increment() {
  count.value = count.value + 1
}
</script>

<template>
  <button @click="increment">Clicked {{ count }} times</button>
</template>
```

Rendered output after two clicks:

HTML

```
<button>Clicked 2 times</button>
```

Inside a template a `ref` is unwrapped automatically, so you write `{{ count }}` and not `{{ count.value }}`. In the script block you always need `.value`.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Mounting on `<body>` or on several elements	Vue expects one root it fully owns	mount a dedicated `<div id="app">`
Keeping state in normal global variables	Vue cannot see the change, the view stays stale	use `data()`, `ref()` or `reactive()`
Editing the DOM yourself (`el.textContent = ...`)	the next render overwrites your edit	change state and let the template render
Arrow functions for `methods` in the options API	`this` is not the component instance	use ordinary functions inside option objects
Storing every value in one big root `data()` blob	unrelated changes re-render together	keep state local to the component that uses it
Expecting components to auto-sync with parents	data flows down, events go up	pass props down, emit events up

1.5 Practice exercises

1. Mount an application on `<div id="root">` that renders a `greeting` field from `data()`.
2. Add a `reset()` method that sets the counter back to `0` and wire it to a second button.
3. Rewrite the counter from example 2 using `<script setup>` and `ref`.
4. In one sentence each, explain why `document.querySelector('h1').textContent = 'x'` fights with Vue.

Solutions

1. `createApp({ data() { return { greeting: 'Hi' } } }).mount('#root')` with `<h1>{{ greeting }}</h1>` inside the div.
2. Add `reset() { this.count = 0 }` to `methods`, then a `<button @click="reset">Reset</button>`.
3. `vue <script setup> import { ref } from 'vue' const count = ref(0) const increment = () => count.value++ </script> <template> <button @click="increment">Clicked {{ count }} times</button> </template>`
4. The next time reactive state changes, Vue re-renders from its own record of the DOM and the manual edit disappears; state is the single source of truth, so manual DOM writes are lost work.

1.6 Mini test

1. Multiple choice: what does `mount('#app')` do? - a) replaces the contents of the `#app` element with the rendered component - b) deletes the `#app` element and creates a new one - c) moves the component into a `<script>` tag - d) reloads the page until `#app` appears
2. True or false: one application can control two separate DOM trees at once.
3. Multiple choice: which API creates reactive state in the Composition API? - a) `data()` - b) `methods()` - c) `computed()` - d) `ref()`
4. Short code question: write the two calls that create an application from a component named `App` and mount it on `#app`.
5. True or false: a parent passes data to a child through props and hears back through emitted events.
6. Multiple choice: what is the quickest way to try Vue without any build step? - a) `npm create vue@latest` - b) a `<script>` tag with the global CDN build - c) installing Docker - d) compiling templates by hand
7. Short answer: name two things a component owns besides its template.

Answer key and explanations

1. a - mounting renders the root component in place of the contents of the given element.
2. False - `mount()` takes one container and returns no second mount for another.
3. d - `ref()` is the Composition API factory for reactive state; `data()` and `methods()` belong to the options API.
4. `createApp(App).mount('#app')` - `createApp` builds the application, `mount` attaches it.
5. True - props down, events up is the default data flow.
6. b - the global build needs nothing but a browser.
7. Any two of: state (`data` or `ref`/`reactive`), methods, computed properties, watchers, lifecycle hooks, child components.

Chapter 2 - Template syntax and directives

2.1 Summary

- Templates are HTML plus a small set of `v-` directives.
- `{{ }}` interpolation prints one single JavaScript expression.
- Directives are special attributes that apply reactive behaviour to an element.
- `v-bind` and its `:` shorthand pass values into attributes; `v-on` and its `@` shorthand attach listeners.
- `v-if` removes and recreates elements; `v-show` only toggles a CSS `display` value.
- `v-for` renders lists and always wants a unique `:key` per item.
- `v-once`, `v-pre` and `v-cloak` control how and when the template is compiled and updated.
- `v-html` renders raw HTML, which makes it dangerous with untrusted text.

- Template expressions must be free of side effects; heavy logic belongs in computed values or methods.
- Keep templates declarative - if a template is hard to read, the logic should move to the script.

2.2 Concepts

Interpolation

Text interpolation uses double curly braces and accepts any single JavaScript expression:

HTML

```
<p>{{ price * quantity }}</p>
<p>{{ title.toUpperCase() }}</p>
<p>{{ isReady ? "Ready" : "Waiting" }}</p>
```

What is not allowed inside the braces: statements (`if`, `for`), variable declarations, assignments, or several expressions. `null` and `undefined` render as an empty string. Anything computed while rendering should be pure, because the expression runs again on every re-render.

Anatomy of a directive

output

```
v-on:click.prevent="save"
 |   |    |       |
 |   |    |       +--- value: one expression
 |   |    +----------- modifier: changes behaviour
 |   +---------------- argument: what the directive works on
 +-------------------- name after the v- prefix
```

The shorthands you will use all day

Full form	Shorthand	Meaning
`v-bind:href="url"`	`:href="url"`	bind an attribute to an expression
`v-bind="{ id: someId }"`	`v-bind="obj"`	bind many attributes from an object
`v-on:click="handler"`	`@click="handler"`	attach an event listener

Conditional rendering: `v-if` versus `v-show`

	`v-if`	`v-show`
What it does	adds or removes the element from the DOM	toggles `display: none`
Cost	rendering and destroying real elements	one style write
Has `v-else`?	yes	no
Use when	the block appears rarely	visibility toggles often

	v-if	v-show
Also available	v-else-if, v-if on <template>	one directive only

Use `v-if` when the condition is unlikely to change, and `v-show` when you switch often but the markup is cheap. If the content must not exist in the DOM at all (passwords, private notes), only `v-if` guarantees that.

List rendering

`v-for` iterates arrays and objects. The alias order matters:

output

```
v-for="(item, index) in items"      array: value, then index
v-for="(value, key) in scores"      object: value, then key, then index
```

You can also put `v-for` on a `<template>` tag to render a group of elements per item without leaving a wrapper element behind. Every item needs a `:key` - a stable, unique value from the data. An array index is a poor key when the list can be reordered, because Vue then reuses the wrong component state.

Event handling

The value of `v-on` is either an inline expression or the name of a method:

HTML

```
<button @click="count = count + 1">plus</button>
<button @click="save">save</button>
```

Inline handlers receive the native event object as `$event`: `@click="save('draft', $event)"`. Methods declared in the component are bound for you, so `this` works as expected.

Modifier	Effect
.stop	call event.stopPropagation()
.prevent	call event.preventDefault()
.self	fire only when the event target is the element itself
.once	run the handler one time only
.passive	attach with { passive: true } where supported
.capture	listen in the capture phase instead of bubbling

Key modifiers listen for named keys: `.enter`, `.tab`, `.delete`, `.esc`, `.space`, `.up`, `.down`, `.left`, `.right`, plus system keys `.ctrl`, `.alt`, `.shift`, `.meta` and the exact-matching modifier `.exact`.

Directives for special text

Directive	Behaviour
v-text="msg"	same as {{ msg }}, set as text

Directive	Behaviour
`v-html="html"`	render the string as HTML - sanitise first
`v-once`	render once, ignore later updates
`v-pre`	print the raw mustaches, compile nothing inside
`v-cloak`	stays until mounting; hides the uncompiled flash

2.3 Examples

Example 1: interpolation with expressions.

HTML

```html
<div id="app">
  <p>{{ title.toUpperCase() }} - {{ 2 + 3 }} items</p>
  <p>Subtotal: {{ price * quantity }}</p>
</div>
```

JavaScript

```javascript
Vue.createApp({
  data() {
    return { title: "invoice", price: 8, quantity: 4 };
  }
}).mount("#app");
```

Rendered output:

HTML

```html
<p>INVOICE - 5 items</p>
<p>Subtotal: 32</p>
```

Example 2: binding attributes.

HTML

```html
<div id="app">
  <a :href="url" :target="blank ? '_blank' : '_self'">documentation</a>
  <button :disabled="isSending">Send</button>
</div>
```

JavaScript

```javascript
Vue.createApp({
  data() {
    return {
      url: "https://vuejs.org",
      blank: true,
      isSending: true
    };
  }
}).mount("#app");
```

Rendered output:

HTML

```
<a href="https://vuejs.org" target="_blank">documentation</a>
<button disabled>Send</button>
```

Boolean attributes such as `disabled` are added or removed automatically when the bound expression is truthy or falsy.

Example 3: conditional rendering.

HTML

```
<div id="app">
  <p v-if="status === 'online'">Server reachable.</p>
  <p v-else-if="status === 'slow'">Server is slow.</p>
  <p v-else>Server is down.</p>
  <p v-show="detailsVisible">Details here.</p>
</div>
```

JavaScript

```
Vue.createApp({
  data() {
    return { status: "slow", detailsVisible: false };
  }
}).mount("#app");
```

Rendered output:

HTML

```
<p>Server is slow.</p>
<p style="display: none;">Details here.</p>
```

`v-if` rendered only the branch that matched. `v-show` rendered its paragraph too, but with `display: none` in its style.

Example 4: rendering a list.

HTML

```
<ul id="app">
  <li v-for="(item, index) in items" :key="item.id">
    {{ index + 1 }}. {{ item.name }} - {{ item.price }} EUR
  </li>
</ul>
```

JavaScript

```
Vue.createApp({
  data() {
    return {
      items: [
        { id: 1, name: "apples", price: 2 },
        { id: 2, name: "pears", price: 3 }
      ]
    };
  }
}).mount("#app");
```

Rendered output:

HTML

```
<ul id="app">
  <li>1. apples - 2 EUR</li>
  <li>2. pears - 3 EUR</li>
</ul>
```

The same pattern works for objects: `<p v-for="(value, key) in scores">{{ key }}: {{ value }}</p>` on `{ ada: 3, alan: 5 }` renders `ada: 3` and `alan: 5` as two paragraphs.

Example 5: handling events with modifiers.

HTML

```
<div id="app">
  <button @click="count = count + 1">Clicked {{ count }} times</button>
  <button @click.once="save('draft')">Save once</button>
  <a :href="url" @click.prevent="save('docs')">Documentation</a>
</div>
```

JavaScript

```
Vue.createApp({
  data() {
    return { count: 0, url: "https://vuejs.org" };
  },
  methods: {
    save(kind) {
      console.log("saved:", kind);
    }
  }
}).mount("#app");
```

Console output after clicking the link:

output

```
saved: docs
```

The browser never navigates because `.prevent` calls `preventDefault()`. The `save('draft')` button works only on the first click because of `.once`.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Statements inside `{{ }}`	the template fails to compile	one expression; move logic to computed or methods
`v-for` without `:key`	warnings and wasteful DOM patching	always bind a unique key
`:key="index"` on a reorderable list	rows reuse the wrong state	use a stable id from the data
`v-if` and `v-for` on the same element	runs per item and reads badly	filter in a computed value first
`v-html` with user-typed text	cross-site scripting	text interpolation, or sanitise the input

Mistake	Why it hurts	Best practice
Heavy computation in a template expression	reruns on every render	a computed property caches it
Two statements in one handler	only one expression is allowed	call a method instead

2.5 Practice exercises

1. Write the interpolation that prints `fullName` in uppercase next to a price formatted with a euro sign, using `price` and nothing else.
2. Render `tags: ['vue', 'router', 'pinia']` as list items with correct keys.
3. Bind the `src` and `alt` attributes of an `<img>` from `imgUrl` and `imgAlt`.
4. Write a button that calls `remove(index)` with the row index, and explain why the form around it needs `@submit.prevent`.

Solutions

1. {{ fullName.toUpperCase() }} - {{ price }} EUR - two expressions, no logic.
2. `<li v-for="(tag, index) in tags" :key="index">{{ tag }}</li>`; for a static list the index is an acceptable key, and the value renders directly.
3. `<img :src="imgUrl" :alt="imgAlt">`.
4. `<button @click="remove(index)">x</button>` inside `v-for="(item, index) in items"`; without `.prevent` the form submit reloads the page and the handler never finishes.

2.6 Mini test

1. Multiple choice: which interpolation is valid? - a) `{{ let x = 2; x }}` - b) `{{ price * 2 }}` - c) `{{ if (ok) 'yes' }}` - d) `{{ count = count + 1 }}`
2. Multiple choice: what is the shorthand for `v-bind:href="url"`? - a) `@href="url"` - b) `:href="url"` - c) `#href="url"` - d) `bind:href="url"`
3. True or false: `v-show` removes the element from the DOM while the condition is false.
4. Multiple choice: why does `v-for` require `:key`? - a) it styles the list - b) it gives each node an identity so updates patch the right rows - c) it sorts the items - d) it is needed only for numbers
5. Short code question: write the two template lines that render `items` as `<li>` elements with the name, using a stable key.
6. True or false: `@click.prevent="save"` stops the default browser action for the click.
7. Multiple choice: which directive renders a string as raw HTML? - a) `v-text` - b) `v-raw` - c) `v-html` - d) `v-out`

Answer key and explanations

1. b - interpolation takes one expression; statements and assignments are rejected.

2. b - `:attr` is shorthand for `v-bind:attr`.
3. False - `v-show` only sets `display: none`; `v-if` removes the node.
4. b - keys let Vue match existing DOM to data instead of recreating rows.
5. `<li v-for="item in items" :key="item.id">{{ item.name }}</li>` with an id field as the key.
6. True - `.prevent` calls `preventDefault()` for you.
7. c - `v-html` sets `innerHTML`, so untrusted content must be sanitised first.

Chapter 3 - Reactivity: ref() and reactive()

3.1 Summary

- Reactivity means a change to a value automatically re-runs everything that read it.
- `ref()` wraps any value, including primitives, and you read or write it through `.value`.
- `reactive()` turns a whole object into a deep reactive proxy without any `.value`.
- Templates unwrap refs automatically; scripts must always use `.value`.
- `ref()` is deep by default, so nested objects it holds are reactive too.
- `reactive()` accepts objects, arrays, `Map` and `Set`, but never a primitive.
- Destructuring a `reactive()` object breaks its reactivity; `toRefs()` fixes that.
- `shallowRef()` tracks only replacement of the value, not changes inside it.
- Array mutation works naturally: `push`, `splice` and index assignment are all tracked.
- DOM updates are batched: use `nextTick()` when you need the updated DOM immediately.

3.2 Concepts

What reactivity buys you

output

```
plain JavaScript                reactive state
+----------------+              +--------------------+
| let count = 0  |  change it   | const count = ref(0)|
|                | ----------->  | count.value = 1    |
| view unchanged |              | template re-renders |
+----------------+              +--------------------+
```

A plain variable cannot tell anyone that it changed. Vue wraps reactive values in objects that record who read them, so when the write happens every reader is scheduled for another run - computed values, watchers, and component renders.

`ref()` and `reactive()` side by side

	`ref()`	`reactive()`
Accepts	any value, including numbers and strings	objects, arrays, `Map`, `Set` only

	ref()	reactive()
Access in script	x.value	x.prop
Reassignment	x.value = 2	mutate properties; do not replace the object
Depth	deep by default	deep by default
Keeps identity when passed around	yes	no - the proxy is bound to one variable

`.value` and unwrapping

output

```
1  template:  {{ count }}          unwrapped automatically
2  script:    count.value          always .value
3  inside a reactive object:       state.someRef   -> unwrapped as well
```

A `ref` stored as a property of a `reactive()` object is unwrapped like a normal property, so you read `state.name` even if `state.name` was created with `ref()`.

The limits of `reactive()`

Three rules cause most confusion:

1. Destructuring takes a plain copy: `let { count } = state` copies the number, and later writes to the copy never reach the object.
2. Replacing the whole object (`state = reactive({...})` again) keeps the old proxy alive everywhere it was passed.
3. It needs an object argument; there is nothing to proxy for a primitive.

Use `toRefs(state)` when you want individual reactive references, and prefer `ref()` when the whole value is replaced rather than mutated.

Shallow and frozen variants

Function	Behaviour
ref(v)	deep tracking
shallowRef(v)	tracks only x.value = ...
reactive(obj)	deep tracking for objects
shallowReactive(obj)	tracks only top-level property changes
readonly(obj)	blocks writes - useful for props you must not change

`shallowRef` is the right tool for a large blob you replace wholesale, such as the response of an API call.

Batching and `nextTick()`

State changes do not write to the DOM immediately. They are queued and flushed together after the current synchronous code finishes, so ten changes in a row cost one render. `nextTick()` returns a promise that resolves once the flush has happened.

3.3 Examples

Example 1: a `ref` in a plain script.

JavaScript

```
import { ref } from "vue";

const count = ref(0);
console.log(count.value);

count.value = count.value + 1;
console.log(count.value);
```

output

```
0
1
```

Example 2: a `reactive` object with arrays.

JavaScript

```
import { reactive } from "vue";

const state = reactive({
  title: "todo",
  tags: ["work", "home"]
});

state.title = "shopping list";
state.tags.push("garden");
state.tags[0] = "chores";

console.log(state.title);
console.log(state.tags.join(", "));
```

output

```
shopping list
chores, home, garden
```

Example 3: template unwrapping. The component needs the dev server from Chapter 10.

Vue

```
<script setup>
import { ref } from "vue"

const price = ref(10)
const quantity = ref(3)
</script>

<template>
  <p>{{ price }} EUR x {{ quantity }}</p>
</template>
```

Rendered output:

HTML

```
<p>10 EUR x 3</p>
```

No `.value` is needed inside the template - Vue unwraps top-level refs for you.

Example 4: destructuring breaks, `toRefs` repairs.

JavaScript

```
import { reactive, toRefs } from "vue";

const state = reactive({ count: 0, name: "Ada" });

let { count } = state;        // plain copy of the number 0
count = 99;
console.log(state.count, count);

const fields = toRefs(state);
fields.count.value = 99;     // writes through the proxy
console.log(state.count);
```

output

```
0 99
99
```

Example 5: `shallowRef` tracks replacement only.

JavaScript

```
import { shallowRef, watch } from "vue";

const raw = shallowRef({ hits: 0 });

watch(raw, () => console.log("changed"), { flush: "sync" });

raw.value.hits = 1;        // in-place change: not tracked
raw.value = { hits: 1 };   // replacement: tracked

console.log("done");
```

output

```
changed
done
```

`flush: "sync"` makes the callback run the moment the value is replaced, which is why `"changed"` appears before `"done"`.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Reading a `ref` without `.value` in script	you compare or assign the wrapper object	use `.value` outside templates

Mistake	Why it hurts	Best practice
`reactive(0)` for a primitive	nothing is wrapped, updates never fire	`ref()` for primitives
Destructuring a `reactive()` object	plain copies stop updating	`toRefs()` or keep `state.x`
Replacing a whole `reactive()` object	other holders keep the old proxy	mutate properties, or switch to `ref()`
Deep tracking a huge structure	unnecessary work on every change	`shallowRef` for replace-wholesale data
Expecting fresh DOM right after a change	updates are batched	`await nextTick()` first
Writing to props inside a child	the parent re-renders and clobbers it	emit an event instead

3.5 Practice exercises

1. Rewrite a counter so that `console.log` prints the value after one increment, using `ref()`.
2. A teammate wrote `const state = reactive(0)` and wondered why nothing updates. Explain and fix.
3. `const { name } = reactive({ name: 'Ada' })` does not update. Show the two ways to keep the connection.
4. Write a `shallowRef` line that replaces a large payload, and say what is not tracked.

Solutions

1. `js import { ref } from "vue"; const count = ref(0); count.value = 1; console.log(count.value); // 1`
2. `reactive()` only wraps objects, so the number is passed through untouched and Vue cannot intercept writes to it; use `ref(0)` and `count.value = 1`.
3. Either keep it as `state.name`, or `const fields = toRefs(state); fields.name.value = 'Alan'` - both write through the reactive proxy.
4. `const list = shallowRef(await fetchPage())` then `list.value = await fetchPage()` - only the second line triggers views; mutating the page object in place does not.

3.6 Mini test

1. Multiple choice: how do you read a `ref` in script code? - a) directly, like a variable - b) with `.value` - c) with `ref.get()` - d) with `state.value`
2. True or false: `reactive(42)` creates a reactive number.
3. Multiple choice: what does `const { count } = reactive({ count: 0 })` give you? - a) a reactive reference to `count` - b) a plain copy of the number - c) an error - d) a computed property

4. Short code question: declare a `ref` named `open` with the value `true` and set it to `false`.
5. True or false: `push()` on a `reactive()` array is tracked by Vue.
6. Multiple choice: which helper turns `reactive({ a: 1, b: 2 })` into separate refs? - a) `unref()` - b) `toRef()` - c) `toRefs()` - d) `shallowRef()`
7. Short answer: why does the DOM not change immediately after you write to state?

Answer key and explanations

1. b - `.value` unwraps or writes the inner value in script code.
2. False - `reactive()` returns the value unchanged for primitives because there is nothing to proxy.
3. b - destructuring copies the current value and drops the reactive link.
4. `const open = ref(true)` then `open.value = false`.
5. True - in Vue 3 collection mutators and index writes both trigger updates.
6. c - `toRefs()` creates one ref per property; `toRef()` converts a single one.
7. Updates are queued and flushed together after the current code; `await nextTick()` waits for that flush.

Chapter 4 - Computed properties and watchers

4.1 Summary

- A computed property derives a value from other state and caches the result.
- The cache survives until one of the dependencies changes, then it recomputes.
- A method call runs on every render; a computed property runs only when needed.
- A computed property can also expose a setter, which makes it writable.
- A watcher runs side-effect code when its source changes - it may be async.
- `watch()` accepts a ref, a getter function, an array of sources, or a reactive object.
- Options control timing and depth: `immediate`, `deep`, and `flush`.
- Callbacks of `watch()` receive the new value and the previous one.
- `watchEffect()` tracks whatever it reads during its run, with no explicit source.
- Use computed values for derived data and watchers for effects such as fetching, logging or saving.

4.2 Concepts

Computed property versus method

	Computed	Method
Runs	only when a dependency changes	on every re-render
Result	cached until a dependency changes	new every time
Use for	derived data	actions that must always run

Derived value, cached

output

```
count = ref(3)
        |
        +--> computed(() => count.value * 2)  --> cached 6
             count.value = 4                  --> recomputed 8
             read again                       --> still 8, no rerun
```

A computed property is declared with a getter. It can also take a `get`/`set` object: the setter receives the new value and is expected to write to the source state, which gives you a writable derived value.

Watching state

`watch(source, callback, options)` - what `source` can be:

Source	Example
a `ref`	`watch(query, cb)`
a getter function	`watch(() => props.id, cb)`
a reactive object	`watch(state, cb)` - implicitly deep
an array of the above	`watch([a, b], cb)`

Options worth knowing:

Option	Effect
`immediate: true`	run the callback once right away
`deep: true`	also react to nested changes
`flush: 'sync'`	run while the state is being changed
`flush: 'pre'`	default - run before component updates
`flush: 'post'`	run after the DOM has been patched

Callbacks receive `(newValue, oldValue)`; with several sources both arguments become arrays.

`watchEffect()`

`watchEffect(cb)` runs `cb` once immediately and records every reactive value it touches. When any of them changes, the callback runs again. There is no old value, and the source list cannot be narrowed down - whatever was read is tracked. It suits cases like "run this query

whenever the URL term or the page number changes".

Choosing the right tool

output

```
need a value?        -> computed
need a reaction?     -> watch (explicit sources) or watchEffect (auto-tracked)
need the DOM after?  -> flush: 'post' or await nextTick()
```

4.3 Examples

Example 1: caching, proven with a counter inside the getter.

JavaScript

```
import { ref, computed } from "vue";

const calls = [];
function factorial(n) {
  calls.push(n);
  return n <= 1 ? 1 : n * factorial(n - 1);
}

const n = ref(3);
const result = computed(() => factorial(n.value));

console.log(result.value);          // computes 1,2,3
console.log(result.value);          // cached, no new pushes
console.log(calls.join(","));       // 3,2,1

n.value = 4;
console.log(result.value);          // recomputes
console.log(calls.join(","));       // 3,2,1,4,3,2,1
```

output

```
6
6
3,2,1
24
3,2,1,4,3,2,1
```

Example 2: a writable computed property.

JavaScript

```js
import { ref, computed } from "vue";

const firstName = ref("Ada");
const lastName = ref("Lovelace");

const fullName = computed({
  get: () => `${firstName.value} ${lastName.value}`,
  set: (value) => {
    const parts = value.split(" ");
    firstName.value = parts[0];
    lastName.value = parts[1];
  }
});

console.log(fullName.value);
fullName.value = "Alan Turing";
console.log(firstName.value, lastName.value);
```

output

```
Ada Lovelace
Alan Turing
```

Example 3: `watch` with `immediate`.

JavaScript

```js
import { ref, watch, nextTick } from "vue";

const question = ref("");

watch(
  question,
  (q) => console.log("answer for:", q || "(empty)"),
  { immediate: true }
);

question.value = "life";
console.log("synchronous point");

nextTick().then(() => console.log("after flush"));
```

output

```
answer for: (empty)
synchronous point
answer for: life
after flush
```

The immediate call happens at once; the re-run waits for the microtask flush, which is why the log appears after the synchronous point.

Example 4: `watchEffect` auto-tracks its dependencies.

JavaScript

```
import { ref, watchEffect } from "vue";

const width = ref(2);
const height = ref(3);

watchEffect(() => {
  console.log("area =", width.value * height.value);
});

width.value = 4;
```

output

```
area = 6
area = 12
```

The callback runs once when created, then again because `width` was read inside it during that run.

Example 5: waiting for the DOM inside a component.

Vue

```
<script setup>
import { ref, nextTick } from "vue"

const count = ref(0)
const el = ref(null)

async function increment() {
  count.value = count.value + 1
  console.log("before:", el.value.textContent)
  await nextTick()
  console.log("after:", el.value.textContent)
}
</script>

<template>
  <p ref="el">{{ count }}</p>
  <button @click="increment">plus</button>
</template>
```

Console output after one click:

output

```
before: 0
after: 1
```

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Async work inside a computed getter	caches a pending promise	fetch in a watcher or an action instead
Expecting a watcher to run at once	by default it is lazy	add `immediate: true`

Mistake	Why it hurts	Best practice
Watching only part of an object	nested changes are missed	watch the object itself or use `deep: true`
Writing the watched value inside its own callback	infinite loop	compare new and old values before writing
DOM reads inside a default watcher	the DOM is not patched yet	`flush: 'post'` or `await nextTick()`
Leaving a watcher alive after a component is gone	memory leak	stop it in `onUnmounted`, or create it in setup
Recomputing pure derivation in a watcher	no cache, more renders	a computed property

4.5 Practice exercises

1. Write a computed property named `discounted` that is `price * 0.9`, and log it twice without recomputation happening twice.
2. Turn `quantity` and `price` into a total with a writable computed property that accepts a new number and adjusts `quantity`.
3. Write a `watch` on a `query` ref that logs the trimmed value, and make it run once at creation.
4. Why is `watchEffect` a bad match for a callback that needs the previous value?

Solutions

1. `js import { ref, computed } from "vue"; const price = ref(100); const discounted = computed(() => price.value * 0.9); console.log(discounted.value); console.log(discounted.value); // cached, computed getter runs once`
2. `js import { ref, computed } from "vue"; const price = ref(10); const quantity = ref(2); const total = computed({ get: () => price.value * quantity.value, set: (v) => { quantity.value = Math.round(v / price.value); } });`
3. `js import { ref, watch } from "vue"; const query = ref(""); watch(query, (q) => console.log(q.trim()), { immediate: true });`
4. `watchEffect` never passes values to its callback; it re-runs the whole effect body instead, so there is nothing to compare the new value against. Use `watch` when you need old and new values.

4.6 Mini test

1. Multiple choice: when does a computed property re-evaluate? - a) on every render - b) only when a dependency changes - c) every millisecond - d) never after the first read
2. True or false: a method call is cached like a computed property.
3. Multiple choice: which option makes a watcher run once when it is created? - a) `deep: true` - b) `immediate: true` - c) `flush: 'sync'` - d) `once: true`

4. Short code question: declare a computed property `double` for a `count` ref.
5. True or false: `watchEffect` runs its callback once when it is created.
6. Multiple choice: what do the two arguments of a `watch` callback receive? - a) the event and the element - b) the new value and the previous value - c) the ref and the proxy - d) nothing at all
7. Short answer: which tool do you use to wait for the DOM to be patched inside a component?

Answer key and explanations

1. b - the cached value is invalidated only when a tracked dependency writes.
2. False - a plain method runs on every render; only computed values cache.
3. b - `immediate: true` invokes the callback right away.
4. `const double = computed(() => count.value * 2);`
5. True - the first run both executes the effect and records its dependencies.
6. b - `(newValue, oldValue)` for a single source, arrays when there are several.
7. `nextTick()` (or `flush: 'post'`), because updates are batched until the current code finishes.

Chapter 5 - Class, style bindings and transitions

5.1 Summary

- `:class` and `:style` are ordinary bindings with special object and array syntaxes.
- Object syntax for `:class` turns a map of `class: condition` into class names.
- Array syntax for `:class` lists classes and can mix in the object form.
- Object syntax for `:style` maps CSS properties to values, camelCase or quoted kebab-case.
- Array syntax for `:style` merges several style objects; class and style attributes merge too.
- Numbers for unit-carrying properties such as `top` or `fontSize` get the unit added for you.
- The `<Transition>` component animates one element entering or leaving the DOM.
- Six transition classes are applied in a fixed order, driven by CSS or JavaScript hooks.
- `<TransitionGroup>` animates lists, including moving the survivors into their new places.
- `mode="out-in"` sequences two animations when one element replaces another.

5.2 Concepts

Class binding

output

```
:class="{ active: isActive, 'text-danger': hasError }"
:class="[isActive ? 'active' : 'idle', { 'text-danger': hasError }]"
:class="computedClassObject"
```

A static `class="btn"` and a bound `:class` are merged by Vue, so you can keep base classes in the plain attribute and conditional ones in the binding.

Style binding

Syntax	Example
object, camelCase	`:style="{ color: textColor, fontSize: size + 'px' }"`
object, kebab-case	`:style="{ 'font-weight': 700 }"` (keys with a dash must be quoted)
array of objects	`:style="[baseStyle, errorStyle]"` (later entries win)
several values	an array per property for vendor fallbacks; the browser uses the one it supports

Numeric values receive `px` automatically for properties that expect a length; properties such as `fontWeight` or `zIndex` stay unitless, and properties starting with `--` are treated as custom properties and written with `setProperty`.

The `<Transition>` classes

output

```
enter:  v-enter-from  ->  v-enter-active  ->  v-enter-to  ->  removed
leave:  v-leave-from  ->  v-leave-active  ->  v-leave-to  ->  removed
```

With `name="fade"` the classes become `fade-enter-from`, `fade-enter-active`, `fade-enter-to` and the three `fade-leave-*` names. Vue reads the computed `transition-duration` of the active class, so keep the timing in CSS and let Vue clean up the classes. Add `appear` to animate on the first render.

Transitions need a single child

`<Transition>` wraps one element or component. Two simultaneous children are not animated. When two states replace each other in the same slot, use `mode="out-in"` (leave first, then enter) or the default `mode="in-out"`.

`<TransitionGroup>`

- Renders a real element (`tag="ul"` by default `span`) containing the list.
- Every child still needs its `:key`.
- Adding the `v-move` class (`<name>-move`) animates the remaining items when the list reorders, because Vue measures their positions before and after.

5.3 Examples

Example 1: class binding.

HTML

```
<div id="app">
  <p :class="{ active: isActive, 'text-danger': hasError }">line one</p>
  <p :class="[isActive ? 'active' : 'idle', { 'text-danger': hasError }]">line two</p>
  <p :class="computedClasses">line three</p>
</div>
```

JavaScript

```
Vue.createApp({
  data() {
    return {
      isActive: true,
      hasError: false,
      computedClasses: ["active", "shadow"]
    };
  }
}).mount("#app");
```

Rendered output:

HTML

```
<p class="active">line one</p>
<p class="active">line two</p>
<p class="active shadow">line three</p>
```

Example 2: style binding.

HTML

```
<div id="app">
  <p :style="{ color: textColor, fontSize: size + 'px' }">styled</p>
  <p :style="{ 'font-weight': weight }">bold</p>
  <p :style="[baseStyle, errorStyle]">merged</p>
</div>
```

JavaScript

```
Vue.createApp({
  data() {
    return {
      textColor: "crimson",
      size: 20,
      weight: 700,
      baseStyle: { color: "navy" },
      errorStyle: { border: "1px solid red" }
    };
  }
}).mount("#app");
```

Rendered output:

HTML

```
<p style="color: crimson; font-size: 20px;">styled</p>
<p style="font-weight: 700;">bold</p>
<p style="color: navy; border: 1px solid red;">merged</p>
```

`fontWeight` stays unitless, `fontSize` received `px` because the expression supplied it. Only `border` from `errorStyle` survives because `baseStyle` has no `border` of its own.

Example 3: a fade transition.

Vue

```vue
<script setup>
import { ref } from "vue"

const show = ref(true)
</script>

<template>
  <button @click="show = !show">{{ show ? "Hide" : "Show" }}</button>
  <Transition name="fade">
    <p v-if="show">Now you see me</p>
  </Transition>
</template>

<style scoped>
.fade-enter-from,
.fade-leave-to {
  opacity: 0;
}
.fade-enter-active,
.fade-leave-active {
  transition: opacity 0.3s ease;
}
.fade-leave-active {
  position: absolute;
}
</style>
```

Classes applied over time:

output

```
enter:  fade-enter-from  ->  fade-enter-active  ->  fade-enter-to
leave:  fade-leave-from  ->  fade-leave-active  ->  fade-leave-to
```

The element is removed from the DOM only after the leave transition ends, and `position: absolute` keeps the rest of the layout from jumping while it fades.

Example 4: animating a reorder with `<TransitionGroup>`.

Vue

```
<script setup>
import { ref } from "vue"

const items = ref([1, 2, 3, 4, 5])

function shuffle() {
  items.value = items.value.slice().sort(() => Math.random() - 0.5)
}
</script>

<template>
  <button @click="shuffle">Shuffle</button>
  <TransitionGroup name="list" tag="ul" class="rows">
    <li v-for="item in items" :key="item">{{ item }}</li>
  </TransitionGroup>
</template>

<style scoped>
.list-move,
.list-enter-active,
.list-leave-active {
  transition: all 0.5s ease;
}
.list-enter-from,
.list-leave-to {
  opacity: 0;
}
</style>
```

Rendered output before any click:

HTML

```
<ul class="rows">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
</ul>
```

After clicking Shuffle the survivors receive the `list-move` class and glide from their old positions to the new ones, because Vue measures both layouts.

Example 5: `v-show` and `:class` in the same template.

HTML

```
<div id="app">
  <button @click="panelOpen = !panelOpen">Toggle</button>
  <div v-show="panelOpen" :class="{ panel: true, open: panelOpen }">panel</div>
</div>
```

JavaScript

```
Vue.createApp({
  data() {
    return { panelOpen: false };
  }
}).mount("#app");
```

Rendered output while closed:

HTML

```html
<div style="display: none;" class="panel">panel</div>
```

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Expecting `:class` to replace `class="btn"`	they are merged, not replaced	base classes stay static, toggles go to `:class`
Unquoted kebab-case style keys	a syntax error in the object literal	`"font-size"` or camelCase `fontSize`
Passing an object where a string is expected	`class="[object Object]"` appears	remember what each expression returns
Expecting a number to become `px` everywhere	only length properties get the unit	append the unit yourself for certainty
Several children inside one `<Transition>`	only the first is animated	one child, or sequence with `mode`
Missing `:key` in `<TransitionGroup>`	warnings and no move animation	always key the children
Long animations for large lists	janky frames on slow devices	keep durations short and test on a phone

5.5 Practice exercises

1. Bind `active` and `big` classes with a single `:class` using `isActive` and `isBig`.
2. Write a `:style` binding that sets `color` from `textColor` and a `20px` font size.
3. Explain the class order Vue applies when an element enters the DOM inside `<Transition>`.
4. A list reorder does not animate even though you have a `<TransitionGroup>`. Give two likely causes.

Solutions

1. `:class="{ active: isActive, big: isBig }"`.
2. `:style="{ color: textColor, fontSize: '20px' }"` - or `fontSize: size + 'px'` if the number lives in state.
3. `v-enter-from`, then `v-enter-active` (the timing class), then `v-enter-to`; Vue removes them when the transition finishes.
4. The children have no `:key` (Vue cannot match them for the move animation), and/or no `-move` class with a `transition` property (`list-move` for `name="list"`).

5.6 Mini test

1. Multiple choice: which binding turns booleans into class names? - a) `:class="[isActive]"` - b) `:class="{ active: isActive }"` - c) `class={{ isActive }}` - d) `v-class="isActive"`
2. Multiple choice: what happens when a bound `:class` meets a static `class="btn"`? - a) the static one wins - b) the bound one replaces it - c) Vue merges both - d) an error is thrown
3. True or false: `:style="{ 'font-size': 20 }"` must quote the property name.
4. Short code question: write a `:class` binding that adds `active` when `isActive` is true and `idle` otherwise.
5. Multiple choice: which class does Vue add while an element is leaving a `<Transition name="fade">`? - a) `fade-enter-from` - b) `fade-leave-active` - c) `fade-out` - d) `fade-hidden`
6. True or false: every child of `<TransitionGroup>` needs a `:key`.
7. Short answer: name the class that animates the position of the remaining items when a list reorders.

Answer key and explanations

1. b - object syntax maps class names to conditions.
2. c - static and dynamic classes are always combined.
3. True - a dashed key is not a valid JavaScript identifier, so quotes are required.
4. `:class="[isActive ? 'active' : 'idle']"` (or `:class="{ active: isActive, idle: !isActive }"`).
5. b - `fade-leave-active` carries the transition timing; `fade-leave-from` and `fade-leave-to` carry the styles.
6. True - keys give each item an identity so moves and removals can be animated.
7. The move class: `list-move` (or `<name>-move` in general).

Chapter 6 - Forms with v-model

6.1 Summary

- `v-model` creates a two-way binding between a form control and a piece of state.
- On a text input it sets the value and listens for `input` events.
- A checkbox binds a boolean; several checkboxes sharing one binding collect an array.
- A radio button needs a `value` attribute so the options can be told apart.
- A select uses the `value` of the chosen `<option>` and can be `multiple`.
- Bind non-string values with `:value` so numbers and objects survive.
- Modifiers `.trim`, `.number` and `.lazy` change how and when the value is read.
- On a component, `v-model` means the `modelValue` prop plus the `update:modelValue` event.
- A custom control stays controlled by accepting a value prop and emitting updates.
- Never write to a prop from inside a child - emit and let the parent own the state.

6.2 Concepts

What each control binds

Control	Template	State type
text input	`<input v-model="name">`	string
textarea	`<textarea v-model="bio"></textarea>`	string
checkbox	`<input type="checkbox" v-model="agreed">`	boolean
checkbox group	`<input type="checkbox" value="a" v-model="picks">`	array
radio	`<input type="radio" value="a" v-model="picked">`	the chosen `value`
select	`<select v-model="size">` with `<option value="s">`	option value
multi select	`<select multiple v-model="sizes">`	array

Checkboxes accept `true-value` and `false-value` attributes when the checked and unchecked values should not be `true` and `false`.

Modifiers

Modifier	Effect
`.trim`	strip surrounding whitespace
`.number`	parse the text as a number when it looks like one
`.lazy`	sync on the `change` event instead of on every keystroke

The same sugar, written by hand

output

```
<input v-model="name">          ==   <input :value="name" @input="name = $event.target.value">
<Custom v-model="name">         ==   <Custom :modelValue="name" @update:modelValue="name = $event">
```

`v-model` is only a shortcut, and the component form is the reason a custom control can take part in the same pattern.

A controlled custom component

output

```
parent:  state -- modelValue --> child :value
parent:  state <-- update:modelValue <-- child @input
```

The child never stores the text; it echoes the prop and reports keystrokes.

6.3 Examples

Example 1: text fields with `.trim` and `.lazy`.

HTML

```
<div id="app">
  <p>Hello, {{ name }}!</p>
  <input v-model.trim="name" placeholder="your name">
  <p>{{ draft }}</p>
  <textarea v-model.lazy="draft"></textarea>
</div>
```

JavaScript

```
Vue.createApp({
  data() {
    return { name: "", draft: "" };
  }
}).mount("#app");
```

Rendered output after typing `Ada` into the name box and `hello world` into the textarea:

HTML

```
<p>Hello, Ada!</p>
<p>hello world</p>
```

`.trim` removed the spaces from the bound value; `.lazy` kept the textarea in sync only when it lost focus.

Example 2: checkboxes.

HTML

```
<div id="app">
  <label><input type="checkbox" v-model="agreed"> I agree</label>
  <label><input type="checkbox" value="vue" v-model="stack"> Vue</label>
  <label><input type="checkbox" value="router" v-model="stack"> Router</label>
  <p>{{ stack.join(", ") }}</p>
</div>
```

JavaScript

```
Vue.createApp({
  data() {
    return { agreed: false, stack: [] };
  }
}).mount("#app");
```

Rendered output with both stack boxes ticked:

HTML

```
<p>vue, router</p>
```

The single checkbox stored `true` in `agreed`; the shared array collected the values.

Example 3: radios and a select.

HTML

```
<div id="app">
  <label><input type="radio" value="basic" v-model="plan"> Basic</label>
  <label><input type="radio" value="pro" v-model="plan"> Pro</label>
  <select v-model="size">
    <option value="s">Small</option>
    <option value="m">Medium</option>
  </select>
  <p>{{ plan }} / {{ size }}</p>
</div>
```

JavaScript

```
Vue.createApp({
  data() {
    return { plan: "basic", size: "m" };
  }
}).mount("#app");
```

Rendered output before touching anything:

HTML

```
<p>basic / Medium</p>
```

Each radio writes its own `value` into `plan`; the select writes the `value` of the chosen option into `size`.

Example 4: the `.number` modifier.

HTML

```
<div id="app">
  <input v-model.number="age" type="number">
  <p>{{ typeof age }}: {{ age }}</p>
</div>
```

JavaScript

```
Vue.createApp({
  data() {
    return { age: 0 };
  }
}).mount("#app");
```

Rendered output after typing `30`:

HTML

```
<p>number: 30</p>
```

Without `.number` the binding would keep the characters as a string and `age + 1` would produce `301`.

Example 5: `v-model` on a component.

Vue

```vue
<!-- CustomInput.vue -->
<script setup>
defineProps({ modelValue: String })
const emit = defineEmits(["update:modelValue"])
</script>

<template>
  <input
    :value="modelValue"
    @input="emit('update:modelValue', $event.target.value)"
  >
</template>
```

Vue

```vue
<!-- Parent.vue -->
<script setup>
import { ref } from "vue"
import CustomInput from "./CustomInput.vue"

const name = ref("")
</script>

<template>
  <CustomInput v-model="name" />
  <p>{{ name }}</p>
</template>
```

Rendered output after typing `Ada`:

HTML

```html
<input>
<p>Ada</p>
```

The child is a plain controlled input: it renders the prop and emits every keystroke upward.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Forgetting `.number` on numeric fields	`age + 1` concatenates strings	`.number`, or cast with `Number()`
Two `v-model`s on one control	one binding silently overwrites the other	one piece of state per control
A radio group without `value` attributes	the options are indistinguishable	always give each option a `value`
Writing to the `modelValue` prop inside the child	the parent re-renders and reverts it	emit `update:modelValue`
Expecting `v-model` on a `<div>`	only form elements and components support it	use a component for custom widgets
Strings for numbers bound with `:value`	radio values become strings	bind `:value="42"` for numbers

Mistake	Why it hurts	Best practice
Handling the same edit in parent and child	sources of truth multiply	keep the state in the parent, or in a store

6.5 Practice exercises

1. Bind a checkbox to `subscribe` and print `on` or `off` in a paragraph from its value.
2. Build a two-checkbox group that collects into `langs` and shows the count.
3. Write a custom `Toggle` component that supports `v-model` with a boolean.
4. Why does `<input v-model="age">` with `age` used in `age + 1` produce a surprise, and what are the two fixes?

Solutions

1. `<input type="checkbox" v-model="subscribe">` with `<p>{{ subscribe ? 'on' : 'off' }}</p>`.
2. `<input type="checkbox" value="js" v-model="langs">` and `<input type="checkbox" value="ts" v-model="langs">` with `<p>{{ langs.length }}</p>`.
3. `vue <script setup> defineProps({ modelValue: Boolean }) const emit = defineEmits(["update:modelValue"]) </script> <template> <button @click="emit('update:modelValue', !modelValue)"> {{ modelValue ? "on" : "off" }} </button> </template>`
4. The binding is a string, so `age + 1` concatenates `30` and `1` into `301`; add the `.number` modifier, or convert with `Number(age)` before arithmetic.

6.6 Mini test

1. Multiple choice: what does the `.lazy` modifier change? - a) whitespace handling - b) string to number parsing - c) syncing on `change` instead of every keystroke - d) event throttling
2. Multiple choice: how do several checkboxes collect into one array? - a) each one gets its own `v-model` - b) they share one `v-model` and differ by their `value` - c) they must share an `id` - d) with `v-bind` only
3. True or false: `v-model` on a component maps to the `modelValue` prop and the `update:modelValue` event by default.
4. Short code question: write the two lines a custom input needs to support `v-model`.
5. Multiple choice: which modifier parses numeric input into a number? - a) `.trim` - b) `.number` - c) `.lazy` - d) `.cast`
6. True or false: a child component may assign to its own `modelValue` prop to update the parent.
7. Short answer: spell out what `v-model="name"` compiles to on a plain `<input>`.

Answer key and explanations

1. c - `.lazy` listens for the `change` event rather than each `input` event.

2. b - the shared model is an array and `value` supplies each entry.
3. True - that is the default convention the component form of `v-model` uses.
4. `:value="modelValue"` and `@input="emit('update:modelValue', $event.target.value)"`.
5. b - `.number` converts the string when it parses cleanly.
6. False - props are read-only; the child emits `update:modelValue` instead.
7. `:value="name"` plus `@input="name = $event.target.value"` - the component form swaps in the prop and event names.

Chapter 7 - Components and props

7.1 Summary

- A component is a reusable block with its own template, state and behaviour.
- Single-file components are imported and then used in templates like HTML tags.
- `defineProps` declares what a component accepts from the outside.
- Props flow one way: from parent to child, and a child never writes to them.
- A prop declaration can be a type, an array of types, or an object with validation.
- Validation supports `required`, `default` and a custom `validator` function.
- Attributes a component does not declare land on its root element through `$attrs`.
- `inheritAttrs: false` turns that off so a wrapper can place the attributes itself.
- `defineExpose` decides what a parent may reach through a template ref.
- Naming: file and import in PascalCase; templates accept PascalCase and kebab-case.

7.2 Concepts

Anatomy of a component

output

```
+--------------------------------+
| <script setup>                 |  state, props, logic
| <template>                     |  what it renders
| <style scoped>                 |  styles limited to this component
+--------------------------------+
```

Components nest: a root component renders child components, which render their own children, until you have a tree that matches the interface.

Declaring props

JavaScript

```
// type shorthand
defineProps({ title: String, likes: Number })

// object with validation
defineProps({
  title: { type: String, required: true },
  likes: { type: Number, default: 0 },
  size: {
    type: String,
    default: "m",
    validator: (v) => ["s", "m", "l"].includes(v)
  }
})
```

A `default` for `Object` or `Array` must be a factory: `default: () => []`. The declaration above uses runtime syntax; with TypeScript you declare an interface instead (Chapter 12).

One-way data flow

output

```
parent state  -- prop -->  child prop
parent state  <-- event -- child emits
```

When a parent re-renders, props are updated and the child follows. The child signals intent with events; the parent owns the change.

Attribute inheritance

output

```
<MyInput class="extra" data-role="name" id="q">
        |
        v
root element receives id, data-role and merges class
```

`$attrs` holds everything that was not declared as a prop. A wrapper component usually re-binds them with `v-bind="$attrs"` on the element that should get them, after setting `inheritAttrs: false` so nothing falls on the wrapper itself. `class` and `style` always merge instead of replacing.

`defineExpose` and template refs

With `<script setup>` a component is closed by default: the parent cannot read its internals. Whatever the child passes to `defineExpose` becomes the shape of the parent's ref, which is the clean way to offer an imperative API such as `focus()` or `reset()`.

7.3 Examples

Example 1: a card component with props.

Vue

```vue
<!-- GreetingCard.vue -->
<script setup>
defineProps({
  name: { type: String, required: true },
  title: { type: String, default: "friend" }
})
</script>

<template>
  <p>Hello {{ name }} the {{ title }}!</p>
</template>
```

Vue

```vue
<!-- Parent.vue -->
<script setup>
import { ref } from "vue"
import GreetingCard from "./GreetingCard.vue"

const people = ref([
  { id: 1, name: "Ada", title: "engineer" },
  { id: 2, name: "Alan", title: "mathematician" }
])
</script>

<template>
  <GreetingCard
    v-for="person in people"
    :key="person.id"
    :name="person.name"
    :title="person.title"
  />
</template>
```

Rendered output:

HTML

```html
<p>Hello Ada the engineer!</p>
<p>Hello Alan the mathematician!</p>
```

Example 2: a validator that fails.

Vue

```vue
<script setup>
defineProps({
  size: {
    type: String,
    default: "m",
    validator: (v) => ["s", "m", "l"].includes(v)
  }
})
</script>

<template>
  <span class="dot" :class="size"></span>
</template>
```

With `<Badge size="xl" />` the development console prints a warning and the component still renders:

output

```
[Vue warn] Invalid prop: custom validator check failed for prop "size".
```

Example 3: attribute inheritance with `v-bind="$attrs"`.

Vue

```vue
<!-- FancyButton.vue -->
<script setup>
defineOptions({ inheritAttrs: false })
</script>

<template>
  <button class="btn" v-bind="$attrs">Click</button>
</template>
```

Vue

```vue
<!-- Parent.vue -->
<script setup>
import FancyButton from "./FancyButton.vue"

function save() {
  console.log("saved")
}
</script>

<template>
  <FancyButton class="extra" data-role="action" @click="save" />
</template>
```

Rendered output:

HTML

```html
<button class="btn extra" data-role="action">Click</button>
```

The static class of the component merges with the one from the parent, and the listener arrived through `$attrs`.

Example 4: an imperative API with `defineExpose`.

Vue

```vue
<!-- Field.vue -->
<script setup>
import { ref, defineExpose } from "vue"

const input = ref(null)

defineExpose({
  focus: () => input.value.focus()
})
</script>

<template>
  <input ref="input" placeholder="focus me">
</template>
```

Vue

```vue
<!-- Parent.vue -->
<script setup>
import { ref } from "vue"
import Field from "./Field.vue"

const field = ref(null)

function onOpen() {
  field.value.focus()
}
</script>

<template>
  <Field ref="field" />
  <button @click="onOpen">Focus the field</button>
</template>
```

Rendered output before any click:

HTML

```html
<input placeholder="focus me">
<button>Focus the field</button>
```

Clicking the button calls the exposed method and the input receives focus, because the parent only ever reaches the child through `focus`.

Example 5: missing required props warn too.

Vue

```vue
<!-- Avatar.vue -->
<script setup>
defineProps({
  src: { type: String, required: true },
  alt: { type: String, default: "avatar" }
})
</script>

<template>
  <img :src="src" :alt="alt">
</template>
```

With `<Avatar />` the development console prints:

output

```
[Vue warn] Missing required prop: "src"
```

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Assigning to a prop inside the child	the next parent render reverts it	emit an event, or copy into local state with care
`default: []` for arrays and objects	the same instance is shared by every component	`default: () => []`

Mistake	Why it hurts	Best practice
`greeting="42"` expecting a number	the attribute is a string	bind it: `:greeting="42"`
A thick tree of props passed two levels down	hard to follow and refactor	provide/inject or a store for deep trees
Skipping `:key` when components come from `v-for`	patching mistakes	key every repeated component
Relying on `$attrs` reaching a nested element	attributes land on the root	set `inheritAttrs: false` and bind deliberately
Reaching into a child with a ref to read state	breaks encapsulation	`defineExpose` a narrow API

7.5 Practice exercises

1. Declare `label` as a required `String` prop and `color` as a `String` that defaults to `"gray"`.
2. Write the parent template line that passes the `person` object's `name` and `age` fields as props.
3. A `<Panel class="big">` renders its inner div without the `big` class. Explain and fix it with `$attrs`.
4. Give the child a `clear()` method that resets its own `input`, and expose it to the parent.

Solutions

1. `js defineProps({ label: { type: String, required: true }, color: { type: String, default: "gray" } })`
2. `<Card :name="person.name" :age="person.age" />`.
3. `class` merges on the root element, so if the outer element is not the one that needs it, turn inheritance off and bind manually: `defineOptions({ inheritAttrs: false })` then `v-bind="$attrs"` on the inner div.
4. `vue <script setup> import { ref, defineExpose } from "vue" const input = ref(null) function clear() { input.value = "" } defineExpose({ clear }) </script> <template> <input ref="input"> </template>`

7.6 Mini test

1. Multiple choice: how does a child receive data from its parent? - a) through events - b) through props - c) through `$attrs` only - d) by reading the parent's state
2. Multiple choice: where do undeclared attributes end up? - a) nowhere, they are dropped - b) on the component's root element - c) in the console - d) on every child
3. True or false: a component may write directly to one of its own props.
4. Short code question: declare a required `String` prop named `label`.
5. Multiple choice: which default is correct for an `Array` prop? - a) `default: []` - b) `default: () => []` - c) `default: "[]"` - d) `default: null`

6. Multiple choice: which option stops automatic attribute fallthrough? - a) `inheritAttrs: false` - b) `attrs: false` - c) `inheritProps: false` - d) `propsInherit: false`
7. Short answer: what does `defineExpose` control?

Answer key and explanations

1. b - props are the declared inputs; events are the outputs.
2. b - undeclared attributes land on the root element through `$attrs` unless inheritance is disabled.
3. False - writing to a prop is blocked by design; emit an event instead.
4. `defineProps({ label: { type: String, required: true } })`.
5. b - object and array defaults must be factories so every instance gets its own.
6. a - `inheritAttrs: false` (or `defineOptions({ inheritAttrs: false })`) disables it.
7. Which methods and state the parent is allowed to reach through a component ref.

Chapter 8 - Events, slots and provide/inject

8.1 Summary

- Children talk to parents by declaring and emitting events.
- `defineEmits` declares the event list; `emit` fires one with a payload.
- An emit declaration can be an object with validators, like props.
- `v-model` on a component uses the `modelValue` prop and the `update:modelValue` event.
- Named `v-model` bindings, such as `v-model:visible`, swap those two names.
- Slots pass markup from a parent into a child component.
- The default slot sits where `<slot />` is placed; named slots use `#name`.
- Scoped slots hand child data back to the parent's markup.
- Fallback content shows when a slot is never filled.
- `provide` and `inject` share values with descendants without prop drilling.

8.2 Concepts

Events flow upward

output

```
child:  emit("submit", payload)
              |
parent: @submit="onSubmit"   -> onSubmit(payload)
```

The parent decides what a callback does; the child only announces that something happened. A child that emits `update:modelValue` can be driven by `v-model` without any extra code.

`v-model` on a component, spelled out

output

```
<Child v-model="name">        ==  <Child :modelValue="name" @update:modelValue="name = $event">
<Child v-model:visible="open"> ==  <Child :visible="open" @update:visible="open = $event">
```

Any prop name can take part: bind `v-model:size` for `:size` and `@update:size`.

Slots at a glance

Kind	Template	Purpose
default	`<slot />`	whatever the parent wraps around the tag
named	`<slot name="header" />` used with `<template #header>`	named holes
scoped	`<slot name="row" :item="item" />`	let the parent render with child data
fallback	`<slot>Nothing here</slot>`	shows when the hole is empty
presence check	`const slots = useSlots()` then `slots.header`	render optional chrome only when needed

Slot content is compiled in the parent's scope, so it sees parent state, while the placement inside the child decides where it appears.

provide and inject

output

```
Ancestor.provide("key", value)   ->   any descendant.inject("key", defaultValue)
```

Provide can be called in `setup` (visible to that component's subtree) or on the application object (visible everywhere). Injection with a default value never fails; without one it warns and returns `undefined` when nothing was provided. For larger projects prefer a Pinia store, but `provide`/`inject` is perfect for theme, locale or a plugin instance.

8.3 Examples

Example 1: a child that raises the count.

Vue

```vue
<!-- CounterButton.vue -->
<script setup>
defineProps({ modelValue: { type: Number, default: 0 } })
const emit = defineEmits(["update:modelValue"])
</script>

<template>
  <button @click="emit('update:modelValue', modelValue + 1)">
    Clicked {{ modelValue }} times
  </button>
</template>
```

Vue

```vue
<!-- Parent.vue -->
<script setup>
import { ref } from "vue"
import CounterButton from "./CounterButton.vue"

const count = ref(0)
</script>

<template>
  <CounterButton v-model="count" />
  <p>The parent still owns the value: {{ count }}</p>
</template>
```

Rendered output after two clicks:

HTML

```html
<button>Clicked 2 times</button>
<p>The parent still owns the value: 2</p>
```

Example 2: validating an event payload.

Vue

```vue
<!-- SignupForm.vue -->
<script setup>
const emit = defineEmits({
  submit: (payload) => {
    if (!payload || typeof payload.email !== "string") {
      console.warn("submit expects { email }")
      return false
    }
    return true
  }
})

function onSubmit() {
  const ok = emit("submit", { email: "ada@example.com" })
  console.log("accepted:", ok)
}
</script>

<template>
  <button @click="onSubmit">Sign up</button>
</template>
```

Console output after clicking:

output

```
accepted: true
```

The boolean returned by `emit` reflects the validators: `false` when a validator fails, which is why the form can decide whether to continue.

Example 3: named slots with fallbacks.

Vue

```vue
<!-- Panel.vue -->
<script setup>
import { useSlots } from "vue"

const slots = useSlots()
</script>

<template>
  <section class="panel">
    <header v-if="slots.header"><slot name="header" /></header>
    <div class="body"><slot>Nothing to show.</slot></div>
    <footer v-if="slots.footer"><slot name="footer" /></footer>
  </section>
</template>
```

Vue

```vue
<!-- Parent.vue -->
<script setup>
import Panel from "./Panel.vue"
</script>

<template>
  <Panel>
    <template #header><h2>Title</h2></template>
    <p>Body content from the parent.</p>
  </Panel>
</template>
```

Rendered output:

HTML

```html
<section class="panel">
  <header><h2>Title</h2></header>
  <div class="body"><p>Body content from the parent.</p></div>
</section>
```

The footer is absent because its slot was never filled; the `v-if` on `slots.footer` keeps the wrapper from rendering an empty element.

Example 4: a scoped slot.

Vue

```vue
<!-- TodoList.vue -->
<script setup>
defineProps({ items: { type: Array, required: true } })
</script>

<template>
  <ul>
    <li v-for="(item, index) in items" :key="item.id">
      <slot name="row" :item="item" :index="index">{{ item.text }}</slot>
    </li>
  </ul>
</template>
```

Vue

```vue
<!-- Parent.vue -->
<script setup>
import { ref } from "vue"
import TodoList from "./TodoList.vue"

const todos = ref([
  { id: 1, text: "learn vue" },
  { id: 2, text: "build app" }
])
</script>

<template>
  <TodoList :items="todos">
    <template #row="{ item, index }">
      <strong>{{ index + 1 }}.</strong> {{ item.text.toUpperCase() }}
    </template>
  </TodoList>
</template>
```

Rendered output:

HTML

```html
<ul>
  <li><strong>1.</strong> LEARN VUE</li>
  <li><strong>2.</strong> BUILD APP</li>
</ul>
```

Example 5: a theme from `provide`.

Vue

```vue
<!-- ThemeProvider.vue -->
<script setup>
import { provide, ref } from "vue"

const theme = ref("light")

provide("theme", theme)
provide("toggleTheme", () => {
  theme.value = theme.value === "light" ? "dark" : "light"
})
</script>

<template>
  <div :class="theme"><slot /></div>
</template>
```

Vue

```vue
<!-- ThemeButton.vue -->
<script setup>
import { inject } from "vue"

const theme = inject("theme")
const toggleTheme = inject("toggleTheme")
</script>

<template>
  <button @click="toggleTheme">current: {{ theme }}</button>
</template>
```

Vue

```vue
<!-- App.vue -->
<script setup>
import ThemeProvider from "./ThemeProvider.vue"
import ThemeButton from "./ThemeButton.vue"
</script>

<template>
  <ThemeProvider>
    <ThemeButton />
  </ThemeProvider>
</template>
```

Rendered output:

HTML

```html
<div class="light"><button>current: light</button></div>
```

The button is not a child of the provider's script; it still finds both values because `inject` walks up the component tree.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Emitting names that differ from the declaration	the listener never fires and Vue warns	keep one spelling; use `v-model` sugar
Mutating a prop to "send data back"	the parent re-render reverts it	emit `update:x` events
Ignoring the return value of `emit`	failed validation goes unnoticed	let the callback steer the flow
Wrapping everything in `provide`	dependencies become invisible	props first, store for global data
`inject` without a default outside a provider	warning plus `undefined`	always pass a sensible default
Slot prop names that shadow parent variables	confusing to read	use distinctive names such as `item`
Deeply nested slot templates	hard to follow	extract a small component

8.5 Practice exercises

1. Write a `Rating` component that accepts a number and emits `change` with the clicked value.
2. Rewrite `<input :value="name" @input="name = $event.target.value">` with `v-model`.
3. Build a `Card` with a default slot and a named `title` slot, with a fallback title of `"Untitled"`.
4. When is `provide` the better choice over props, and what is the usual alternative once an app grows?

Solutions

1. `vue <script setup> defineProps({ value: { type: Number, default: 0 } }) const emit = defineEmits(["change"]) </script> <template> <button v-for="n in 5" :key="n" @click="emit('change', n)">{{ n }}</button> </template>`
2. `<input v-model="name">`.
3. `vue <template> <div class="card"> <h3><slot name="title">Untitled</slot></h3> <slot /> </div> </template>`
4. When the same value would travel through several levels that do not use it; once the app grows, a Pinia store gives the same reach with explicit, testable state.

8.6 Mini test

1. Multiple choice: which pair implements `v-model` on a component by convention? - a) `value` prop and `input` event - b) `modelValue` prop and `update:modelValue` event - c) `model` prop and `change` event - d) `data` prop and `update` event
2. Multiple choice: how is a named slot filled from the parent? - a) `<slot name="header">` - b) `<template #header>` - c) `slot="header"` - d) `<fill name="header">`
3. True or false: `emit` returns a boolean that reflects whether an emit validator failed.
4. Short code question: declare emits `change` and `delete` in `<script setup>`.
5. True or false: content inside a slot is compiled in the child's scope.
6. Multiple choice: what does `inject("key", fallback)` return when no provider exists? - a) it throws - b) `undefined` - c) the fallback value - d) an empty ref
7. Short answer: which helper tells you whether a named slot received content?

Answer key and explanations

1. b - that is the default convention behind the `v-model` syntax.
2. b - `<template #header>` (or `v-slot:header`) fills the named slot.
3. True - `emit()` returns `false` when a validator rejects the payload.
4. `const emit = defineEmits(["change", "delete"])`.
5. False - slot content is compiled in the parent's scope, so it sees parent data.
6. c - the second argument is the default used when nothing was provided.
7. `useSlots()` - `slots.header` is truthy only when the parent filled it.

Chapter 9 - Lifecycle hooks, template refs and composables

9.1 Summary

- Lifecycle hooks run your code at defined moments of a component's life.

- `onMounted` runs after the DOM is in place - the home of fetch, measurement, third-party widgets.
- Update hooks run after each re-render; unmount hooks clean up what you started.
- Timers, listeners and observers must be stopped in an unmount hook.
- Template refs point at rendered elements or at a child's exposed API.
- Inside `v-for` a plain `ref` attribute does not collect elements; use a function ref.
- A composable is a function that packages reusable stateful logic, named `useX`.
- Composables may call lifecycle hooks, which then belong to the calling component.
- A module-level `ref` inside a composable behaves like shared state.
- Watching or observing outside `setup` needs a manual stop to avoid leaks.

9.2 Concepts

The lifecycle in order

output

```
setup -> onBeforeMount -> onMounted
                       -> onBeforeUpdate -> onUpdated   (repeats on each change)
                       -> onBeforeUnmount -> onUnmounted
extra: onActivated / onDeactivated (inside <KeepAlive>), onErrorCaptured, onServerPrefetch (SSR)
```

Hook	Typical use
`onBeforeMount`	last chance before the first render; rarely needed
`onMounted`	fetch data, measure the DOM, start intervals, focus input
`onBeforeUpdate`	read the DOM before it is patched
`onUpdated`	read the DOM after the patch (pair with `nextTick` when unsure)
`onBeforeUnmount`	clear timers, remove listeners, cancel requests
`onUnmounted`	final teardown
`onErrorCaptured`	report errors from descendants

Template refs

output

```
const el = ref(null)  +  <div ref="el">   -> el.value is the DOM node after mount
const child = ref(null) + <Child ref="child"> -> child.value is what the child exposes
```

For lists use a function ref so every rendered node is collected: `<li v-for="i in n" :key="i" :ref="(el) => { if (el) items.value.push(el) }">`.

Composables

output

```
useCounter()  -> { count, increment }   one responsibility, named useX
```

A composable is an ordinary function that creates state with `ref` or `reactive` and returns what a component needs. It may use lifecycle hooks and watchers, which then run inside whichever component called it. Because modules are evaluated once, a `ref` declared at module level is shared by every component that imports the composable - a small global store.

9.3 Examples

Example 1: mount, update and unmount in one component.

Vue

```vue
<script setup>
import { ref, onMounted, onUpdated, onBeforeUnmount } from "vue"

const seconds = ref(0)
let timer = null

onMounted(() => {
  console.log("mounted")
  timer = setInterval(() => {
    seconds.value = seconds.value + 1
  }, 1000)
})

onUpdated(() => {
  console.log("updated, seconds =", seconds.value)
})

onBeforeUnmount(() => {
  clearInterval(timer)
  console.log("timer cleared")
})
</script>

<template>
  <p>Uptime: {{ seconds }}s</p>
</template>
```

Rendered output shortly after mounting:

HTML

```html
<p>Uptime: 2s</p>
```

Console output around teardown:

output

```
mounted
updated, seconds = 1
updated, seconds = 2
timer cleared
```

Example 2: focusing an input on mount.

Vue

```vue
<script setup>
import { ref, onMounted } from "vue"

const search = ref(null)

onMounted(() => {
  search.value.focus()
})
</script>

<template>
  <input ref="search" placeholder="Search">
</template>
```

Rendered output:

HTML

```html
<input placeholder="Search">
```

The element exists only after mounting, so the focus call happens inside `onMounted`.

Example 3: a composable with independent state per caller.

JavaScript

```javascript
import { ref } from "vue";

function useCounter(start = 0) {
  const count = ref(start);
  const increment = () => {
    count.value = count.value + 1;
  };
  return { count, increment };
}

const a = useCounter(0);
const b = useCounter(10);

a.increment();

console.log(a.count.value, b.count.value);
```

output

```
1 10
```

Each call created its own `count`; touching `a` left `b` alone.

Example 4: a composable with module-level shared state.

JavaScript

```
// useAuth.js
import { ref } from "vue";

const user = ref({ name: "", signedIn: false });

export function useAuth() {
  function signIn(name) {
    user.value = { name, signedIn: true };
  }
  function signOut() {
    user.value = { name: "", signedIn: false };
  }
  return { user, signIn, signOut };
}
```

Vue

```
<!-- Two components that both import useAuth() share one user object. -->
<script setup>
import { useAuth } from "./useAuth.js";

const { user, signIn } = useAuth();
</script>

<template>
  <button @click="signIn('Ada')">Sign in</button>
  <span v-if="user.signedIn">Hello {{ user.name }}</span>
</template>
```

Rendered output before the click:

HTML

```
<button>Sign in</button><span></span>
```

After the click, any other component that read `user` renders `Hello Ada` as well.

Example 5: watchers and the scope that stops them.

JavaScript

```
import { ref, watch, onScopeDispose } from "vue";

const query = ref("");
const stop = watch(query, (q) => console.log("searching:", q), { flush: "sync" });

query.value = "vue";
stop();

query.value = "router";
console.log("done");
```

output

```
searching: vue
done
```

Inside a component `setup`, watchers stop automatically when the component unmounts; outside, keep the stop handle and call it.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Reading `el.value` in `setup`	the element does not exist yet	wait for `onMounted`
DOM work in `onUpdated` without `nextTick`	reads the previous layout	`flush: 'post'` or `nextTick`
Forgetting to clear an interval	timers keep firing after unmount	clear it in `onBeforeUnmount`
String `ref` on items inside `v-for`	only one element is tracked	function refs per item
Reaching into a child's DOM with a ref	breaks encapsulation	`defineExpose` a narrow API
Creating watchers inside event handlers	nobody stops them	create them in `setup`
One giant composable with ten concerns	impossible to reuse	one responsibility per `useX`

9.5 Practice exercises

1. Add an `onMounted` hook that loads `notes` with `fetch` and stores the JSON.
2. Write a `useInterval` composable that ticks every second and clears itself on unmount.
3. Collect all list elements of a `v-for` into an array with a function ref.
4. Explain the difference in state lifetime between a composable's local `ref` and a module-level `ref`.

Solutions

1. `js import { ref, onMounted } from "vue"; const notes = ref([]); onMounted(async () => { const res = await fetch("/api/notes"); notes.value = await res.json(); });`
2. `js import { ref, onBeforeUnmount } from "vue"; export function useInterval(cb, ms = 1000) { const tick = ref(0); const id = setInterval(() => { tick.value = tick.value + 1; cb(); }, ms); onBeforeUnmount(() => clearInterval(id)); return tick; }`
3. `js const items = ref([]);` with `<li v-for="i in 5" :key="i" :ref="(el) => { if (el) items.value.push(el) }">`.
4. A local `ref` is created on each call, so two components get separate state; a module-level `ref` is created once, so every importer shares it.

9.6 Mini test

1. Multiple choice: which hook runs right after the DOM is inserted? - a) `onBeforeMount` - b) `onMounted` - c) `onUpdated` - d) `onCreated`
2. True or false: an interval started in `onMounted` clears itself when the component unmounts.

3. Multiple choice: what is the name convention for a composable? - a) `makeX` - b) `useX` - c) `withX` - d) `createX`
4. Short code question: declare a template ref named `search` for an `<input>`.
5. True or false: a watcher created inside `setup` stops automatically when the component unmounts.
6. Multiple choice: which collection method is needed to gather elements rendered by `v-for`? - a) a string ref on the last item - b) a function ref per item - c) `querySelectorAll` - d) `useRefs()`
7. Short answer: in a composable, what makes state shared across components?

Answer key and explanations

1. b - `onMounted` runs once the component is in the DOM.
2. False - timers must be cleared in `onBeforeUnmount` or `onUnmounted`.
3. b - the `use` prefix is the community and documentation convention.
4. `const search = ref(null)` and `<input ref="search">`.
5. True - effects created in `setup` are bound to the component's effect scope.
6. b - function refs receive each element as it mounts.
7. A `ref` declared at module level, because module code is evaluated once per application.

Chapter 10 - Single-file components and build tooling

10.1 Summary

- A single-file component (`.vue`) keeps template, script and style in one file.
- The build step compiles the template into a render function and rewrites the script.
- `<script setup>` is the idiomatic block: top-level bindings become the template's scope.
- `<style scoped>` adds an attribute so the rules apply only inside the component.
- `<style module>` or a plain `<style>` keep styles global; scoped is the safe default.
- Vite starts a dev server with on-the-fly compilation and hot module replacement.
- `npm create vue@latest` scaffolds a project and offers Router, Pinia, Vitest and ESLint.
- The daily commands are `npm run dev`, `npm run build` and `npm run preview`.
- `index.html` loads `src/main.js`, which mounts the root component.
- The build output is static HTML, CSS and JavaScript that any host can serve.

10.2 Concepts

The three blocks of an `.vue` file

Block	What it holds	Notes
`<template>`	the markup with directives	one root element is still the norm
`<script setup>`	state, imports, logic	bindings are automatically available in the template
`<style scoped>`	component CSS	gets a unique attribute per component

A plain `<script>` block sits next to `<script setup>` only for advanced cases such as synchronising options with the setup scope. `defineOptions` inside `<script setup>` can set component options such as `name` and `inheritAttrs` without a second block.

A project layout

output

```
my-app/
  index.html            entry that loads main.js
  package.json          scripts and dependencies
  vite.config.js        plugins, aliases, server options
  src/
    main.js             createApp(App).mount('#app')
    App.vue             root component
    components/         reusable pieces
    views/              one file per page (with the router)
```

What Vite does for you

Command	Effect
`npm run dev`	dev server with instant recompiles and hot module replacement
`npm run build`	production bundle into `dist/`
`npm run preview`	serves the production build locally
`npm run test:unit`	unit tests with Vitest (when the option was chosen)

Hot module replacement means that after you save a file, only the changed module is swapped; component state usually survives, which keeps the feedback loop tight. Imports of `.vue`, `css` and image files are resolved by Vite, so no manual wiring is needed.

10.3 Examples

Example 1: the smallest real project.

HTML

```
<!-- index.html -->
<!doctype html>
<html>
  <head>
    <title>Counter app</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

JavaScript

```
// src/main.js
import { createApp } from "vue";
import App from "./App.vue";

createApp(App).mount("#app");
```

Vue

```
<!-- src/App.vue -->
<script setup>
import { ref } from "vue";
import CounterButton from "./components/CounterButton.vue";

const count = ref(0);
</script>

<template>
  <h1>Counter</h1>
  <CounterButton v-model="count" />
  <p>Current: {{ count }}</p>
</template>

<style scoped>
h1 {
  color: teal;
}
</style>
```

Rendered output after two clicks of the button inside `CounterButton.vue`:

HTML

```
<h1>Counter</h1>
<button>Clicked 2 times</button>
<p>Current: 2</p>
```

Example 2: scaffolding a project. This is an interactive terminal session, so there is no captured output.

shell

```
npm create vue@latest my-app
cd my-app
npm install
npm run dev
```

The wizard asks about TypeScript, the router, Pinia, Vitest and linting. Say no to everything for a first project; the packages can be added later.

Example 3: scoped styles in the rendered page.

Vue

```vue
<script setup>
</script>

<template>
  <h2 class="title">Hello</h2>
</template>

<style scoped>
.title {
  color: purple;
}
</style>
```

Rendered output (the attribute suffix is generated per component):

HTML

```html
<h2 class="title" data-v-xxxxxxx>Hello</h2>
```

A rule in another component's `<style scoped>` block cannot match that element because the generated attribute differs.

Example 4: component options without a second script block.

Vue

```vue
<script setup>
defineOptions({
  name: "AppHeader",
  inheritAttrs: false
});

defineProps({ sticky: Boolean });
</script>

<template>
  <header :class="{ sticky }"><slot /></header>
</template>
```

Rendered output with `sticky` true and the text `Home` passed in:

HTML

```html
<header class="sticky">Home</header>
```

Example 5: what the build produces.

shell

```shell
npm run build
```

output

```
dist/
  index.html
  assets/
    index-8f3a2c1d.css
    index-4e9b7f2a.js
```

The file names carry a content hash so browsers can cache them for a long time. Any static host - GitHub Pages, Netlify, a plain nginx - can serve the `dist` folder as it is.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Editing the mounted DOM directly	the next render overwrites it	change state and let the template render
Global styles inside `<style scoped>` for app-wide rules	the rules do not leave the component	a plain `<style>` block, or a shared css file
Bindings used in the template before they are declared	compile-time errors	declare every binding at the top level of `<script setup>`
Committing `dist/` and `node_modules/`	noisy repository and merge pain	leave them out with `.gitignore`
Ignoring build warnings	bundle grows and runtime surprises	read them; split heavy imports
Using the CDN build for a large app	no build step, no code splitting, slower	a Vite project from the start

10.5 Practice exercises

1. List the files the scaffold creates under `src/` for a project without Router or Pinia.
2. Write `main.js` for an app whose root component is `App` and which uses a `router`.
3. Explain what `<style scoped>` does to a rule and to the rendered element.
4. Which command produces the folder you can upload to a static host?

Solutions

1. `main.js`, `App.vue`, a `components/` folder (often with a demo component), plus `assets/` for shared styles and images.
2. `js import { createApp } from "vue"; import App from "./App.vue"; import router from "./router"; createApp(App).use(router).mount("#app");`
3. It appends a component-specific attribute to the rendered elements and rewrites each selector with that attribute, so the rule matches only inside this component.
4. `npm run build`, which writes the `dist/` folder.

10.6 Mini test

1. Multiple choice: which block of an `.vue` file holds the component logic? - a) `<template>` - b) `<script setup>` - c) `<style scoped>` - d) `<main>`
2. Multiple choice: which command starts the development server? - a) `npm start` - b) `npm run dev` - c) `npm run serve` - d) `vite run`
3. True or false: `<style scoped>` rules leak into every other component.
4. Short code question: write the two statements that create and mount the application in `main.js`.
5. Multiple choice: what is hot module replacement good for? - a) smaller bundles - b) swapping changed modules without a full page reload - c) compiling templates only - d) removing the need for keys
6. True or false: the file that mounts the application is `index.html`.
7. Short answer: which command creates the static production output?

Answer key and explanations

1. b - `<script setup>` is the idiomatic logic block.
2. b - `npm run dev` runs the Vite dev server.
3. False - scoped rules are rewritten with a unique attribute and stay local.
4. `js import { createApp } from "vue"; import App from "./App.vue"; createApp(App).mount("#app");`
5. b - only the changed module is replaced, so state and window position survive.
6. False - `index.html` only loads `src/main.js`; `main.js` performs the mount.
7. `npm run build`.

Cheat sheet

11.1 Template directives

Directive	Shorthand	Purpose
`v-bind:attr`	`:attr`	bind an attribute to an expression
`v-on:event`	`@event`	attach an event listener
`v-model`	-	two-way bind a form control or component
`v-if` / `v-else-if` / `v-else`	-	add or remove elements
`v-show`	-	toggle `display` only
`v-for`	-	render a list, always with `:key`
`v-html`	-	render raw HTML (sanitise first)
`v-text`	-	set text content
`v-once`	-	render once, never update

Directive	Shorthand	Purpose
`v-pre`	-	print the mustaches verbatim
`v-cloak`	-	hide until mounted

Modifiers: `v-on` `.stop`, `.prevent`, `.self`, `.once`, `.passive`, `.capture`, `.exact` and key modifiers (`.enter`, `.esc`, `.tab`, `.delete`, ...). `v-model`: `.trim`, `.number`, `.lazy`.

11.2 Composition API

Function	Use
`ref(v)`	reactive value, read with `.value`
`reactive(obj)`	deep reactive object
`computed(() => ...)`	cached derived value (also `get`/`set` form)
`watch(src, cb, opts)`	run code on change; `immediate`, `deep`, `flush`
`watchEffect(cb)`	auto-tracked effect, runs once immediately
`toRef` / `toRefs`	keep reactivity when splitting an object
`shallowRef` / `shallowReactive`	top-level tracking only
`readonly`	freeze against writes
`nextTick()`	promise resolved after the DOM flush
`defineProps`	declare component inputs
`defineEmits`	declare component outputs
`defineExpose`	limit what a parent ref can reach
`defineOptions`	set options such as `name`, `inheritAttrs`
`useSlots` / `useAttrs`	inspect slots and attributes

11.3 Components and communication

Pattern	Summary
props	parent to child, one way, read-only in the child
emits	child to parent; `v-model` uses `modelValue` + `update:modelValue`
slots	parent markup inside the child; named, scoped and with fallbacks
provide / inject	share values down the tree without prop drilling
composables	`useX` functions packaging reusable stateful logic
Pinia store	shared state when provide gets too implicit

11.4 Lifecycle

Hook	When
`onBeforeMount`	before first render
`onMounted`	after the DOM insert
`onBeforeUpdate`	before a re-render
`onUpdated`	after the DOM was patched
`onBeforeUnmount`	cleanup timers and listeners here
`onUnmounted`	component is gone
`onActivated` / `onDeactivated`	inside `<KeepAlive>`
`onErrorCaptured`	catch errors from descendants

11.5 Project commands

shell

```shell
npm create vue@latest        # scaffold (router, pinia, vitest, eslint are optional)
npm install                  # install dependencies
npm run dev                  # dev server on http://localhost:5173
npm run build                # production bundle in dist/
npm run preview              # serve the production build
```

11.6 Router essentials

JSREF

```jsref
// src/router/index.js (installed with the router package)
import { createRouter, createWebHistory } from "vue-router";
import HomeView from "../views/HomeView.vue";

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: "/", name: "home", component: HomeView },
    { path: "/todos/:id", name: "todo", component: () => import("../views/TodoView.vue") },
    { path: "/:pathMatch(.*)*", name: "not-found", component: () => import("../views/NotFound.vue") }
  ]
});

// in a component
import { useRoute, useRouter } from "vue-router";
const route = useRoute();         // read params and query
const router = useRouter();       // router.push("/"), router.back()
```

11.7 Pinia essentials

JSREF

```
import { defineStore } from "pinia";

export const useTodosStore = defineStore("todos", {
  state: () => ({ items: [], loading: false }),
  getters: {
    done: (state) => state.items.filter((t) => t.done)
  },
  actions: {
    async load() {
      this.loading = true;
      try {
        const res = await fetch("/api/todos");
        this.items = await res.json();
      } finally {
        this.loading = false;
      }
    },
    add(text) {
      this.items.push({ id: Date.now(), text, done: false });
    }
  }
});
```

JSREF

```
// in a component
import { storeToRefs } from "pinia";
const store = useTodosStore();
const { items, loading } = storeToRefs(store);   // keep reactivity when destructuring
store.add("write tests");
```

Final test

1. Multiple choice: how do you read a `ref` in script code? - a) directly like a variable - b) with `.value` - c) with `ref()` - d) with `state.value`
2. Multiple choice: which directive renders a list of elements? - a) `v-if` - b) `v-for` - c) `v-show` - d) `v-on`
3. True or false: `v-show` keeps the element in the DOM and only sets `display: none`.
4. Multiple choice: by default, `v-model` on a component uses which prop and event? - a) `value` and `input` - b) `modelValue` and `update:modelValue` - c) `model` and `change` - d) `data` and `update`
5. Short code question: declare a computed property named `double` for a `count` ref.
6. Multiple choice: which watcher option runs the callback once at creation? - a) `deep: true` - b) `immediate: true` - c) `flush: 'sync'` - d) `once: true`
7. True or false: a child component may assign a new value to one of its own props.
8. Multiple choice: where do attributes that a component does not declare end up? - a) they are dropped - b) on the component's root element - c) in `localStorage` - d) on every grandchild
9. Short code question: declare `label` as a required `String` prop and `count` with default `0`.
10. Multiple choice: what does `:key` give every item of a `v-for`? - a) a styling hook - b) a stable identity for patching - c) an index - d) a scope

11. True or false: `watchEffect` runs its callback once when it is created.
12. Multiple choice: which hook runs immediately after the component's DOM is inserted? - a) `onBeforeMount` - b) `onMounted` - c) `onUpdated` - d) `onBeforeUnmount`
13. Short answer: what are the shorthand characters for `v-bind` and `v-on`?
14. Multiple choice: what does `defineExpose` control? - a) which props are public - b) what a parent ref can reach on this component - c) which CSS is scoped - d) the emits list
15. True or false: top-level bindings in `<script setup>` are directly usable in the template.
16. Multiple choice: which command scaffolds a new Vue project? - a) `npm new vue` - b) `npm create vue@latest` - c) `vue init app` - d) `npm start vue-app`
17. Short code question: write the two lifecycle hooks that fetch data after mounting and clear an interval before unmounting.
18. Multiple choice: what does `flush: 'post'` do on a watcher? - a) prevents the callback - b) runs it before component updates - c) runs it after the DOM has been patched - d) runs it only once
19. True or false: content passed into a slot is compiled in the parent's scope.
20. Multiple choice: which binding adds `active` only while `isActive` is true? - a) `:class="'active'"` - b) `:class="{ active: isActive }"` - c) `class="active: isActive"` - d) `v-class="isActive"`
21. Short answer: name one practical difference between a computed property and a method.
22. Multiple choice: which modifier makes `v-model` store a number? - a) `.trim` - b) `.number` - c) `.lazy` - d) `.cast`
23. True or false: `provide` in one component can be read with `inject` in a deeply nested descendant.
24. Multiple choice: what does `storeToRefs` do with a Pinia store? - a) deletes its state - b) converts its state and getters into refs for safe destructuring - c) replaces actions with mutations - d) exports the store as JSON
25. Short code question: write the `emit` call that increases a `modelValue` prop by one inside a `v-model` driven button.

12.1 Final test answers

1. b - outside templates a `ref` is read and written through `.value`.
2. b - `v-for` renders one element per item, with `:key` for identity.
3. True - only `v-if` adds or removes elements; `v-show` toggles the style.
4. b - the default convention is the `modelValue` prop plus the `update:modelValue` event.
5. `const double = computed(() => count.value * 2);`
6. b - `immediate: true` invokes the callback once right away.
7. False - props are read-only; a child emits an event and the parent changes state.
8. b - undeclared attributes fall through to the root element and are available as `$attrs`.

9. `js defineProps({ label: { type: String, required: true }, count: { type: Number, default: 0 } });`
10. b - a key tells Vue which data row matches which DOM node, so patches stay correct.
11. True - the first run both executes the effect and records its dependencies.
12. b - `onMounted` runs after insertion, which is when DOM APIs are safe to use.
13. `:` for `v-bind` and `@` for `v-on`.
14. b - whatever is passed to `defineExpose` is the shape of the parent's template ref.
15. True - the compiler exposes top-level bindings to the template without an object wrapper.
16. b - `npm create vue@latest` runs the official project wizard.
17. `js import { onMounted, onBeforeUnmount } from "vue"; onMounted(() => { /* fetch here */ }); onBeforeUnmount(() => { clearInterval(timer); });`
18. c - `flush: 'post'` schedules the callback after the DOM update is applied.
19. True - slot markup belongs to the parent, so it sees parent state and parent methods.
20. b - object syntax maps class names to conditions and merges with static classes.
21. Either: a computed value is cached and only re-evaluates when a dependency changes, while a method runs on every render; or: a computed is synchronous, so an async method cannot be a computed.
22. b - `.number` converts the input text to a number when it parses cleanly.
23. True - `inject` walks up the component tree, so any descendant can read the value.
24. b - `storeToRefs` wraps state and getters in refs so destructuring keeps reactivity.
25. `emit("update:modelValue", modelValue + 1);`

Sources

All URLs were consulted while writing this volume. The main source is the official Vue documentation; the package sites cover Router, Pinia and the testing utilities, and the two build-tool pages come from the volumes' own toolchain.

- Vue tutorial: https://vuejs.org/tutorial/
- Introduction to Vue: https://vuejs.org/guide/introduction
- Creating an application: https://vuejs.org/guide/essentials/application
- Template syntax: https://vuejs.org/guide/essentials/template-syntax
- Reactivity fundamentals: https://vuejs.org/guide/essentials/reactivity-fundamentals
- Computed properties: https://vuejs.org/guide/essentials/computed
- Class and style bindings: https://vuejs.org/guide/essentials/class-and-style
- Conditional rendering: https://vuejs.org/guide/essentials/conditional
- List rendering: https://vuejs.org/guide/essentials/list
- Event handling: https://vuejs.org/guide/essentials/event-handling
- Form input bindings: https://vuejs.org/guide/essentials/forms
- Component basics: https://vuejs.org/guide/essentials/component-basics

- Props: https://vuejs.org/guide/components/props
- Component events: https://vuejs.org/guide/components/events
- Slots: https://vuejs.org/guide/components/slots
- Provide and inject: https://vuejs.org/guide/components/provide-inject
- Lifecycle hooks: https://vuejs.org/guide/essentials/lifecycle
- Watchers: https://vuejs.org/guide/essentials/watchers
- Template refs: https://vuejs.org/guide/essentials/template-refs
- Composables: https://vuejs.org/guide/reusability/composables
- Tooling and SFCs: https://vuejs.org/guide/scaling-up/tooling
- Transitions: https://vuejs.org/guide/built-ins/transition
- TypeScript with Vue: https://vuejs.org/guide/typescript/overview
- API reference: https://vuejs.org/api/
- Vue Router: https://router.vuejs.org/
- Pinia: https://pinia.vuejs.org/
- Vue test utils: https://test-utils.vuejs.org/
- Vitest: https://vitest.dev/
- Vite: https://vitejs.dev/
- LearnVue articles (free tier, supplementary): https://learnvue.co/articles

Chapter 11 - Vue Router, Pinia and data fetching

14.1 Summary

- Vue Router maps URLs to components and keeps the browser history in sync.
- `createRouter` with `createWebHistory` is the default setup in a Vite project.
- `<RouterLink>` renders links that switch views without a page reload; `<RouterView>` renders the match.
- `useRoute()` reads the current route, and `useRouter()` navigates programmatically.
- Dynamic segments such as `/todo/:id` arrive as `route.params.id`.
- Route components can be lazily imported so the first bundle stays small.
- Navigation guards can block or redirect before a route resolves.
- Pinia stores hold shared state as `state`, `getters` and `actions`.
- `storeToRefs` destructures a store without losing reactivity.
- Fetching belongs in an action or in `onMounted`, always with loading and error states.

14.2 Concepts

Wiring the router

output

```
createRouter({ history, routes }) -> app.use(router) -> <RouterLink> + <RouterView>
```

`routes` is an array of records. A record has a `path`, a `name`, and either a direct `component` or a lazy `component: () => import("...")` that is fetched only when the route is visited. `createWebHistory` gives clean URLs; the server must rewrite unknown paths to `index.html` for that to work in production.

Reading and moving

API	Purpose
`route.params.id`	dynamic segment values
`route.query.page`	query string values as strings
`router.push({ name: "home" })`	navigate programmatically
`router.back()`	history navigation
`RouterLink to="/about"`	declarative navigation

Watch `route.params` when the same component is reused for different records, because props and params do not reset on their own.

Guards

Kind	Where	Use
global `beforeEach`	router setup	authentication, logging
`beforeEnter` on a record	route record	page-specific rules
`onBeforeRouteUpdate`	inside a component	reload data when params change

A guard can return `false` to cancel, return a route location to redirect, or return `true` to continue.

Pinia in one paragraph

output

```
state: the data      getters: derived values       actions: methods, may be async
const store = useTodosStore();   store.items (reactive)   store.add(text) (call)
```

A store is defined once per application and then used by any component, which makes it the natural home for data several screens share. Because state lives in one place, two components reading the same store see the same value.

Fetching with states

output

```
idle -> loading -> data | error
```

Always track `loading` and `error`: the user needs feedback, and a failed request that renders nothing looks like a broken app. Load in an action or in `onMounted`, and guard against responses that arrive after the component is gone or the query has changed.

14.3 Examples

Example 1: the router file and the shell.

JavaScript

```
// src/router/index.js
import { createRouter, createWebHistory } from "vue-router";
import HomeView from "../views/HomeView.vue";

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: "/", name: "home", component: HomeView },
    { path: "/about", name: "about", component: () => import("../views/AboutView.vue") }
  ]
});

export default router;
```

Vue

```
<!-- App.vue -->
<script setup>
import { RouterLink, RouterView } from "vue-router";
</script>

<template>
  <nav>
    <RouterLink to="/">Home</RouterLink>
    <RouterLink to="/about">About</RouterLink>
  </nav>
  <RouterView />
</template>
```

Rendered output while on `/`:

HTML

```
<nav>
  <a href="/">Home</a>
  <a href="/about">About</a>
</nav>
<main><!-- HomeView renders here --></main>
```

`main.js` adds one line: `createApp(App).use(router).mount("#app")`.

Example 2: a dynamic route that reloads on param change.

JavaScript

```
// route record
{ path: "/todo/:id", name: "todo", component: () => import("../views/TodoView.vue") }
```

Vue

```
<!-- TodoView.vue -->
<script setup>
import { useRoute } from "vue-router";
import { ref, watch } from "vue";

const route = useRoute();
const todo = ref(null);

watch(
  () => route.params.id,
  async (id) => {
    const res = await fetch(`/api/todos/${id}`);
    todo.value = await res.json();
  },
  { immediate: true }
);
</script>

<template>
  <article v-if="todo">
    <h2>{{ todo.text }}</h2>
    <p>done: {{ todo.done }}</p>
  </article>
</template>
```

For `/todo/7` with a server response of `{"text": "write the volume", "done": false}` the rendered output is:

HTML

```
<article>
  <h2>write the volume</h2>
  <p>done: false</p>
</article>
```

Without the watcher, navigating from `/todo/7` to `/todo/8` would only change the URL.

Example 3: a global guard.

JavaScript

```
import { useAuthStore } from "../stores/auth";

router.beforeEach((to) => {
  const auth = useAuthStore();
  if (to.meta.requiresAuth && !auth.signedIn) {
    return { name: "sign-in" };
  }
});
```

The route record sets `meta: { requiresAuth: true }`. When the store says nobody is signed in, the guard returns the sign-in location and the navigation is redirected there instead of rendering the protected page.

Example 4: a store and its consumer.

JavaScript

```
// src/stores/counter.js
import { defineStore } from "pinia";

export const useCounterStore = defineStore("counter", {
  state: () => ({ count: 0 }),
  getters: {
    double: (state) => state.count * 2
  },
  actions: {
    increment() {
      this.count = this.count + 1;
    }
  }
});
```

Vue

```
<!-- CounterView.vue -->
<script setup>
import { storeToRefs } from "pinia";
import { useCounterStore } from "../stores/counter";

const store = useCounterStore();
const { count, double } = storeToRefs(store);
</script>

<template>
  <button @click="store.increment()">{{ count }} / {{ double }}</button>
</template>
```

Rendered output after two clicks:

HTML

```
<button>2 / 4</button>
```

The store must be installed first with `createApp(App).use(createPinia())`. `storeToRefs` keeps `count` and `double` reactive; calling the action directly on `store` works without it.

Example 5: loading and error states.

Vue

```vue
<script setup>
import { ref } from "vue";

const items = ref([]);
const loading = ref(false);
const error = ref(null);

async function load() {
  loading.value = true;
  error.value = null;
  try {
    const res = await fetch("/api/items");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    items.value = await res.json();
  } catch (err) {
    error.value = err.message;
  } finally {
    loading.value = false;
  }
}
</script>

<template>
  <button @click="load">Load</button>
  <p v-if="loading">Loading...</p>
  <p v-else-if="error">Failed: {{ error }}</p>
  <ul v-else>
    <li v-for="item in items" :key="item.id">{{ item.text }}</li>
  </ul>
</template>
```

Rendered output while the request is in flight:

HTML

```html
<button>Load</button>
<p>Loading...</p>
```

If the server answers `500 Internal Server Error`, the paragraph becomes `Failed: HTTP 500` and the list stays hidden until the next attempt.

14.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Reading `route.params` once and never again	stale content when only the id changes	watch the param, or use `onBeforeRouteUpdate`
Importing every view eagerly	large first bundle	lazy `component: () => import(...)` per route
Fetching inside a component render	loops and duplicate requests	load in an action or `onMounted`
Ignoring fetch failures	blank screens	show an error state
Destructuring a store's state directly	values stop being reactive	`storeToRefs`

Mistake	Why it hurts	Best practice
Keeping server state in many components	copies disagree	one store, read from it everywhere
Using a store for purely local UI state	needless global coupling	keep local state local

14.5 Practice exercises

1. Write the route records for `/`, `/todo/:id`, and a catch-all not-found page.
2. Load a task whenever `route.params.id` changes, and show its text.
3. Write a Pinia store with `state`, a getter `count`, and an `increment` action.
4. Add a loading flag and an error message to a `fetch` inside `onMounted`.

Solutions

1. `js [ { path: "/", name: "home", component: HomeView }, { path: "/todo/:id", name: "todo", component: TodoView }, { path: "/:pathMatch(.*)*", name: "not-found", component: NotFoundView } ]`
2. `watch(() => route.params.id, async (id) => { const res = await fetch(`/api/todos/${id}`); task.value = await res.json(); }, { immediate: true });`
3. `js export const useCounterStore = defineStore("counter", { state: () => ({ value: 0 }), getters: { double: (s) => s.value * 2 }, actions: { increment() { this.value = this.value + 1 } } });`
4. `js const loading = ref(false); const error = ref(null); onMounted(async () => { loading.value = true; try { const res = await fetch("/api/items"); if (!res.ok) throw new Error(`HTTP ${res.status}`); items.value = await res.json(); } catch (e) { error.value = e.message; } finally { loading.value = false; } });`

14.6 Mini test

1. Multiple choice: which pair of components renders the routing shell? - a) `RouterLink` and `RouterView` - b) `RouterNav` and `RouterPage` - c) `Link` and `View` - d) `Route` and `RouteView`
2. Multiple choice: what does `route.params.id` contain on `/todo/7`? - a) `7` - b) `/todo` - c) `undefined` - d) the whole path string
3. True or false: a lazy route component is downloaded only when its route is first visited.
4. Short code question: write the array item that maps `/todo/:id` to `TodoView` with the name `todo`.
5. Multiple choice: which helper destructures a Pinia store keeping reactivity? - a) `toRefs(store)` - b) `storeToRefs(store)` - c) `reactive(store)` - d) `unref(store)`
6. True or false: a global `beforeEach` guard may return a route location to redirect.
7. Short answer: which three pieces of state should a fetch request always update?

Answer key and explanations

1. a - links navigate, the view renders the matched component.
2. a - dynamic segments arrive as string params; `Number()` if you need arithmetic.
3. True - the arrow function is only evaluated when the route matches.
4. `{ path: "/todo/:id", name: "todo", component: () => import("../views/TodoView.vue") }` (a direct import works too).
5. b - `storeToRefs` wraps state and getters in refs; `toRefs` is for plain reactive objects.
6. True - returning a location redirects; returning `false` cancels the navigation.
7. The data, a loading flag and an error message.

Chapter 12 - Testing, TypeScript and the mini project

15.1 Summary

- Vitest runs the tests; `@vue/test-utils` mounts components outside the browser.
- `mount` returns a wrapper with `find`, `findAll`, `trigger`, `emitted` and `html`.
- Interactions go through `trigger("click")`, followed by awaiting the update.
- Assert on behaviour: rendered text, emitted events, prop changes.
- Watch mode reruns tests on save, which suits a tight loop.
- TypeScript arrives through `<script setup lang="ts">` and typed declarations.
- `defineProps<Props>()` turns an interface into the component's contract.
- `ref<string[]>()` and friends type the state that scripts carry.
- The mini project combines components, props, events, a store and persistence.
- Prefer the smallest structure that works: local state, then a store, tests as you go.

15.2 Concepts

The testing tools

Tool	Role
Vitest	test runner, `describe` / `it` / `expect`, watch mode
`@vue/test-utils`	`mount` components, query and trigger on the wrapper
`flushPromises`	await pending promises such as a fetch call

A test has three parts: arrange (props and state), act (`trigger`), assert (`expect`). Aim for one behaviour per test.

TypeScript in components

Declaration	Example
typed props	`defineProps<{ label: string; count?: number }>()`
defaults with TS	`withDefaults(defineProps<Props>(), { count: 0 })`
typed emits	`defineEmits<{ change: [value: number] }>()`
typed state	`const names = ref<string[]>([])`
computed type	inferred from the getter's return type

Types make props self-documenting, and the compiler catches a missing prop or a wrong event payload before the code runs.

The mini project shape

output

```
src/
  App.vue               state, filtering, persistence
  components/
    TaskForm.vue        input + submit, emits "add"
    TaskList.vue        list, emits "toggle" and "remove"
```

The app exercises everything the volume covers: an SFC project from Chapter 10, reactive state (Chapter 3), a computed list (Chapter 4), `v-model` and events (Chapters 6 and 8), and two child components with validated props (Chapter 7).

15.3 Examples

Example 1: mounting and triggering.

JavaScript

```js
// tests/CounterButton.test.js
import { describe, it, expect } from "vitest";
import { mount } from "@vue/test-utils";
import CounterButton from "../src/components/CounterButton.vue";

describe("CounterButton", () => {
  it("emits the next value on click", async () => {
    const wrapper = mount(CounterButton, { props: { modelValue: 1 } });

    await wrapper.find("button").trigger("click");

    expect(wrapper.emitted("update:modelValue")).toBeTruthy();
    expect(wrapper.emitted("update:modelValue")[0]).toEqual([2]);
  });
});
```

Test run summary:

output

```
Test Files  1 passed (1)
     Tests  1 passed (1)
```

The assertion is on the event, not on the button's text, because the event is the component's contract with its parent.

Example 2: rendering a list from props.

JavaScript

```
// tests/TaskList.test.js
import { it, expect } from "vitest";
import { mount } from "@vue/test-utils";
import TaskList from "../src/components/TaskList.vue";

it("renders one row per task and marks the done ones", () => {
  const wrapper = mount(TaskList, {
    props: {
      tasks: [
        { id: 1, text: "a", done: true },
        { id: 2, text: "b", done: false }
      ]
    }
  });

  expect(wrapper.findAll("li")).toHaveLength(2);
  expect(wrapper.findAll("li")[0].classes()).toContain("done");
});
```

Test run summary:

output

```
Test Files  1 passed (1)
     Tests  1 passed (1)
```

Example 3: a typed component.

Vue

```
<script setup lang="ts">
interface Props {
  label: string;
  count?: number;
}

const props = withDefaults(defineProps<Props>(), {
  count: 0
});

const emit = defineEmits<{
  change: [value: number];
}>();
</script>

<template>
  <button @click="emit('change', props.count + 1)">
    {{ props.label }}: {{ props.count }}
  </button>
</template>
```

The interface documents the contract, `withDefaults` supplies the default, and the emit signature types the payload. A parent that passes `label={42}` is a type error at build time, not a runtime surprise.

Example 4: typed state in a script block.

JavaScript

```
import { ref, computed } from "vue";

const names = ref<string[]>([]);
const selected = ref<string | null>(null);

names.value.push("Ada");

const upper = computed(() => names.value.map((name) => name.toUpperCase()));

console.log(selected.value, names.value.length, upper.value.join(","));
```

output

```
null 1 ADA
```

Without the generics the array would start as `never[]` and the compiler would reject the push.

Example 5: the mini task application.

Vue

```vue
<!-- src/App.vue -->
<script setup>
import { ref, computed, watch, onMounted } from "vue";
import TaskForm from "./components/TaskForm.vue";
import TaskList from "./components/TaskList.vue";

const tasks = ref([
  { id: 1, text: "read chapter 1", done: true },
  { id: 2, text: "write the router", done: false }
]);
const filter = ref("all");

const visible = computed(() => {
  if (filter.value === "open") {
    return tasks.value.filter((task) => !task.done);
  }
  if (filter.value === "done") {
    return tasks.value.filter((task) => task.done);
  }
  return tasks.value;
});

function add(text) {
  tasks.value.push({ id: Date.now(), text, done: false });
}

function toggle(id) {
  const task = tasks.value.find((item) => item.id === id);
  if (task) {
    task.done = !task.done;
  }
}

function remove(id) {
  tasks.value = tasks.value.filter((task) => task.id !== id);
}

onMounted(() => {
  const saved = localStorage.getItem("tasks");
  if (saved) {
    tasks.value = JSON.parse(saved);
  }
});

watch(
  tasks,
  (value) => {
    localStorage.setItem("tasks", JSON.stringify(value));
  },
  { deep: true }
);
</script>

<template>
  <h1>Tasks</h1>
  <TaskForm @add="add" />
```

Vue (continued)

```
  <div class="filters">
    <label><input type="radio" value="all" v-model="filter"> all</label>
    <label><input type="radio" value="open" v-model="filter"> open</label>
    <label><input type="radio" value="done" v-model="filter"> done</label>
  </div>
  <TaskList :tasks="visible" @toggle="toggle" @remove="remove" />
</template>
```

Vue

```vue
<!-- src/components/TaskForm.vue -->
<script setup>
import { ref } from "vue";

const emit = defineEmits(["add"]);
const text = ref("");

function submit() {
  const value = text.value.trim();
  if (!value) {
    return;
  }
  emit("add", value);
  text.value = "";
}
</script>

<template>
  <form @submit.prevent="submit">
    <input v-model="text" placeholder="new task">
    <button type="submit">Add</button>
  </form>
</template>
```

Vue

```vue
<!-- src/components/TaskList.vue -->
<script setup>
defineProps({
  tasks: { type: Array, required: true }
});
const emit = defineEmits(["toggle", "remove"]);
</script>

<template>
  <ul>
    <li v-for="task in tasks" :key="task.id" :class="{ done: task.done }">
      <label>
        <input
          type="checkbox"
          :checked="task.done"
          @change="emit('toggle', task.id)"
        >
        {{ task.text }}
      </label>
      <button @click="emit('remove', task.id)">x</button>
    </li>
  </ul>
</template>

<style scoped>
.done {
  opacity: 0.6;
}
</style>
```

Rendered output on a fresh visit with the default data and the `all` filter:

HTML

```html
<h1>Tasks</h1>
<form>
  <input placeholder="new task">
  <button type="submit">Add</button>
</form>
<div class="filters">
  <label><input type="radio" value="all"> all</label>
  <label><input type="radio" value="open"> open</label>
  <label><input type="radio" value="done"> done</label>
</div>
<ul>
  <li class="done">
    <label><input type="checkbox" checked> read chapter 1</label>
    <button>x</button>
  </li>
  <li>
    <label><input type="checkbox"> write the router</label>
    <button>x</button>
  </li>
</ul>
```

15.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Testing implementation details such as internal refs	tests break on refactors	assert on rendered output and emitted events
Forgetting `await` after `trigger`	assertions run before the re-render	`await wrapper.trigger(...)`
One test covering every branch	failures point everywhere	one behaviour per test, clear names
Skipping the error path in fetches	only the happy path is trusted	assert the error state too
Typing props after writing components	types drift from behaviour	declare the interface first
Storing everything in one component	the file becomes a wall	split on responsibility, like the mini project
Testing only with default data	edge cases slip through	pass props for the interesting cases

15.5 Practice exercises

1. Write a test that clicks a `Reset` button and checks that `emitted("reset")` is truthy.
2. Type a component with props `{ title: string, items?: { id: number }[] }` and a default empty array.
3. In the mini project, where should the filter live, and why not inside `TaskList`?
4. Add an "open" count to the mini project's heading using a computed property.

Solutions

1. `js const wrapper = mount(ResetButton); await wrapper.find("button").trigger("click"); expect(wrapper.emitted("reset")).toBeTruthy();`
2. `vue <script setup lang="ts"> interface Item { id: number } const props = withDefaults(defineProps<{ title: string; items?: Item[] }>(), { items: () => [] }); </script>`
3. It belongs in `App.vue`, because the filter decides which array `App` hands to `TaskList`; keeping it in `TaskList` would mix the presentation with the rule that decides what is presented.
4. `js const openCount = computed(() => tasks.value.filter((task) => !task.done).length);` used as `<h1>Tasks ({{ openCount }} open)</h1>`.

15.6 Mini test

1. Multiple choice: which function mounts a component in a test? - a) `render()` - b) `mount()` - c) `createApp()` - d) `wrap()`
2. Multiple choice: how does a test click a button on the wrapper? - a) `wrapper.click()` - b) `wrapper.find("button").click()` - c) `await wrapper.find("button").trigger("click")` - d)

```
wrapper.emit("click")
```

3. True or false: `expect(wrapper.emitted("add")).toBeTruthy()` checks that the child fired the event.
4. Short code question: name the assertion that checks a wrapped list for three items.
5. Multiple choice: which macro turns an interface into typed props? - a) `defineProps<Props>()` - b) `withProps<Props>()` - c) `typedProps<Props>()` - d) `props.of<Props>()`
6. True or false: refs must be typed with generics in a TypeScript component to be usable in the template.
7. Short answer: why does the mini project keep `filter` in `App.vue` instead of `TaskList.vue`?

Answer key and explanations

1. b - `mount` from `@vue/test-utils` renders a component into a wrapper.
2. c - `trigger` dispatches a real DOM event and returns a promise for the update.
3. True - `emitted()` records the events a component fired during the test.
4. `expect(wrapper.findAll("li")).toHaveLength(3)`.
5. a - `defineProps<Props>()` maps the interface onto the component's props.
6. False - inference works for most cases; generics help when the initial value is empty and the type cannot be inferred.
7. The filter is a rule about which tasks are shown, which is the parent's decision; `TaskList` should stay a dumb renderer driven by the `tasks` it receives.